

UNIVERSITÀ DEGLI STUDI DI CASSINO E DEL LAZIO MERIDIONALE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELL'INFORMAZIONE "MAURIZIO SCARANO"

CORSO DI LAUREA IN INGEGNERIA INFORMATICA E DELLE TELECOMUNICAZIONI

Sviluppo di un Ambiente in Realtà Aumentata per Estrazione Dati da un Dispositivo Robotico per Riabilitazione Medica

Relatore:
Prof. Filippo Arrichiello

Candidato:
Mattia Iollo
Matricola: 0078078

Correlatore:
Prof. Daniele Di Vito

Anno accademico 2025/2026

Indice

1 Introduzione **5**

1.1 Teoria della Realtà Virtuale e Simulazione Immersiva 5

1.1.1 Il Virtuality Continuum di Milgram e Kishino (1994) . . 6

1.1.2 Le Sei Classi dei Sistemi di Visualizzazione di Milgram e Kishino 7

1.2 Sistemi Virtuali e Robotici nella Riabilitazione Medica 8

1.3 Problematiche e Sfide Ingegneristiche 8

1.4 Inquadramento del Progetto: Il Digital Twin in Realtà Aumentata Spaziale 9

1.5 Organizzazione del Lavoro di Tesi e Risultati Conseguiti 10

2 Sistema di Visualizzazione e Sviluppo in Realtà Aumentata **12**

2.1 Infrastruttura Hardware: Meta Quest 3 e Tecnologia Passthrough 12

2.2 Strumenti di Sviluppo Software ed Ambiente di Emulazione . . 15

2.2.1 L'Ambiente Unity e i Pacchetti Fondamentali 15

2.2.2 Meta Quest Link Application 16

2.2.3 Meta XR Simulator ed Emulazione del Visore 17

2.3 Architettura di Ricezione Dati e Protocollo UDP 17

2.3.1 Motivazioni Ingegneristiche per la Scelta del Protocollo: Analisi Comparativa tra UDP e TCP 18

2.3.2 Implementazione del Socket Ricevitore C# in Unity . . 19

3 Architettura e Controllo del Sistema Robotico **20**

3.1 Infrastruttura Hardware: Il Manipolatore KUKA LBR Med 14 R820 20

3.2 Ambiente di Sviluppo e Librerie Native(Sunrise.Workbench med) 21

3.3 Pianificazione della Traiettoria e Controllo del Moto 22

3.4 Gestione dei Flussi Dati di Telemetria e Intenzione di Moto . . 24

3.5 Trasmissione della Telemetria e Serializzazione del Payload JSON 25

3.5.1 Vantaggi del Formato JSON su Socket UDP 25

4 Progettazione e Sviluppo dell'Applicazione in Realtà Aumentata **26**
4.1 Introduzione al Progetto 26
4.2 Richiami sulle Trasformazioni Rigide 27
4.3 Ancoraggio Spaziale e Calibrazione tramite Marker QR CODE 28
4.4 Sistemi di Riferimento e Rototraslazione 32
4.5 Implementazione del Gateway Python e del Ricevitore UDP . . 35
4.6 Rendering del Digital Twin, Tracciamento della Flangia e Traiettorie Predittive 40
4.7 Interfaccia Utente (HMI) e Tracciamento delle Mani 44

5 Validazione e Criticità Riscontrate **48**
5.1 Ambiente di Simulazione e Collaudo della Catena Dati 48
5.2 Verifica dell'Ancoraggio Spaziale e Calibrazione dell'Offset . . . 49
5.3 Analisi della Sincronizzazione Geometrica e Problematiche Riscontrate 50

6 Conclusioni e Sviluppi Futuri **51**
6.1 Sviluppi Futuri 52

Elenco delle figure

1.1 Virtuality Continuum Secondo Milgram 6

2.1 Visore Meta Quest 3 12
2.2 EWK Secondo Milgram 14
2.3 Status Indicator LED. 14
2.4 Bystander Signalling LED. 15
2.5 Panoramica Generale all'interno di Unity 15
2.6 Panoramica Generale all'interno di Meta Horizon Link 16
2.7 Panoramica Generale all'interno di Meta XR Simulator 17
2.8 Catena Dati Lato Visore 18

3.1 KUKA LBR Med 14 R820 20
3.2 smartPAD Utilizzato dal Dispositivo Robotico 21
3.3 Panoramica all'interno di SunriseWorkspace 21
3.4 Catena Dati Lato KUKA 25

4.1 QR CODE posizionato alla base del KUKA 29
4.2 Conversione Coordinate Robot-Unity 33
4.3 Visuale all'interno di Unity nella sezione del RobotRoot 35
4.4 Digital Twin e Traiettoria Virtuale 41
4.5 Arco di Circonferenza 43
4.6 HMI 47

5.1 Errore mappatura coordinate 50

6.1 Confronto Sistema da Prospettive Differenti 52
6.2 Meta VR Glasses 53

Elenco degli algoritmi

4.1 Raccolta dei campioni del marker (`MarkerManager.cs`). 30
4.2 Funzioni di media e di allineamento (`MarkerManager.cs`). . . . 31
4.3 Ciclo principale del gateway Python. 36
4.4 Ricezione su thread secondario, lato produttore. 38
4.5 Svuotamento della coda, lato consumatore. 39
4.6 Ricostruzione dell'arco circolare a partire da tre punti. 42
4.7 Rilevamento del pinch e commutazione dell'interfaccia. 45
4.8 Orientamento del pannello HMI verso l'operatore. 46

Capitolo 1

Introduzione

1.1 Teoria della Realtà Virtuale e Simulazione Immersiva

Negli ultimi anni, la continua evoluzione delle tecnologie informatiche e la crescente disponibilità di potenza di calcolo grafico di vari dispositivi elettronici hanno portato a una profonda trasformazione nei modelli di interazione uomo-macchina (*Human-Computer Interaction)* [1]. Alla base di questa trasformazione si colloca la teoria della simulazione tridimensionale guidata dal calcolatore, definita come la capacità di generare mondi sintetici digitali dinamici in cui l'utente può interagire mediante flussi di informazione visiva, acustica e tattile [2]. Per comprendere la natura di questi ambienti immersivi, la comunità scientifica fa riferimento a due teorie fondamentali della percezione e dell'apprendimento: la teoria della presenza (*presence theory*) e la teoria dell'apprendimento esperienziale (*experiential learning theory*) [3]. La Teoria della Presenza definisce il grado di illusione psicologica e percettiva per cui l'utente si sente fisicamente collocato all'interno dell'ambiente generato o modificato dal computer. L'immersione visiva e il tracciamento dei movimenti della testa a sei gradi di libertà (*6 DoF*) consentono di aggiornare la prospettiva in tempo reale, generando risposte comportamentali e decisionali simili a quelle del mondo reale [3]. La Teoria dell'Apprendimento Esperienziale, invece, formalizzata da David Kolb, stabilisce che l'apprendimento e il consolidamento delle competenze risultano efficaci al massimo quando l'individuo è direttamente coinvolto in un ciclo attivo di esperienza, riflessione, concettualizzazione e sperimentazione. La simulazione tridimensionale offre una piattaforma ideale per l'apprendimento esperienziale, consentendo agli operatori di esercitarsi e testare procedure complesse all'interno di un contesto controllato e privo di rischi [3].

I concetti legati al senso di presenza e alla percezione spaziale non si limitano agli ambienti interamente virtuali, bensì si estendono attraverso un ampio

spettro di tecnologie visive e interattive. Al fine di fornire una classificazione rigorosa di tali modelli, Paul Milgram e Fumio Kishino (1994) hanno teorizzato il noto **Virtuality Continuum (VC)** [1].

1.1.1 Il Virtuality Continuum di Milgram e Kishino (1994)

Il **Virtuality Continuum (VC)** rappresenta un modello teorico che formalizza uno spettro continuo, lungo il quale vengono classificate le varie realtà partendo dall'ambiente fisico fino alla Realtà Virtuale.

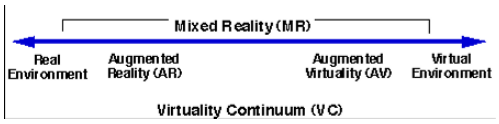


Figura 1.1: Rappresentazione del Virtuality Continuum (VC) secondo Milgram [1]

Lo spettro rappresenta quindi le varie realtà in cui gli estremi opposti mostrati in figura 1.1 sono la realtà fisica pura e l'astrazione digitale totale [1]; in particolare, procedendo da sinistra verso destra si ha:

- **Real Environment (RE):** Estremo sinistro del continuum, rappresenta l'ambiente fisico reale osservato direttamente o tramite ripresa video primaria [1, 4].
- **Mixed Reality (MR)**: Rappresenta il dominio intermedio del continuum in cui sono raggruppati tutti i contesti in cui gli oggetti reali e virtuali coesistono nello stesso campo visivo [1]. La Mixed Reality racchiude al suo interno l'**Augmented Reality (AR)**, dove l'ambiente primario è reale, ma viene "aumentato" attraverso la sovrapposizione di elementi virtuali (es. ologrammi, testi, grafici, ecc...) [1, 4], e l'**Augmented Virtuality (AV)**, dove l'ambiente primario è virtuale in cui vengono proiettati elementi dinamici del mondo reale in tempo reale [1, 4].
- **Virtual Environment (VE/VR):** Estremo destro del continuum, comunemente chiamato Virtual Reality, rappresenta l'ambiente sintetico puro in cui l'utente è completamente immerso e isolato dal mondo esterno [1, 4].

Esiste un'ulteriore definizione più specifica per quanto riguarda l'Augmented Reality (AR), cioè la **Realtà Aumentata Spaziale (Spatial AR)**. Seguendo gli studi di Ronald Azuma (1997), la Realtà Aumentata Spaziale (Spatial AR) si posiziona all'interno del continuum, in particolare, nella porzione in cui la realtà fisica rimane la componente prevalente [1, 2]. Essa non sostituisce l'ambiente reale, al contrario, lo arricchisce inserendo degli ologrammi 3D.

Affinché un sistema possa essere definito come Realtà Aumentata Spaziale, deve rispettare determinate caratteristiche, le quali comprendono: **fondere reale e virtuale**, dove l'utente deve poter vedere simultaneamente sia il mondo fisico che gli elementi digitali; **essere in tempo reale**, dove le interazioni tra l'utente, il robot e gli ologrammi non devono avere ritardi; tutto deve accadere istantaneamente, e infine, deve **essere ancorata allo spazio**, cioè gli ologrammi non devono essere posizionati a caso nello spazio, ma devono essere vincolati a coordinate specifiche del mondo reale e rimanere fisse in quella posizione anche se l'operatore effettua movimenti con la testa [1].

1.1.2 Le Sei Classi dei Sistemi di Visualizzazione di Milgram e Kishino

Oltre alla classificazione del continuum, nella trattazione di Milgram e Kishino (1994) viene effettuata un'ulteriore classificazione riguardante i dispositivi di visualizzazione ibridi all'interno della Realtà Mista, in particolare essi identificano **Sei classi Fondamentali di Ambienti di visualizzazione** [1]. Queste classi si differenziano tra loro in base al canale di visualizzazione e al grado di immersività dell'utente [1], in particolare si ha:

1. **Classe 1 – Monitor-based AR (*Window-on-the-World*,WoW):** Sono sistemi non immersivi in cui immagini grafiche generate dal calcolatore vengono sovrapposte a un flusso video reale bidimensionale [1].

2. **Classe 2 – Immersive Video Displays (HMD basati su Video):** Sono dei sistemi analoghi alla Classe 1 per quanto riguarda l'origine del video reale e la sovrapposizione grafica, ma questi ultimi sono in formato stereoscopico (3D), quindi fruiti tramite un visore immersivo [4].

3. **Classe 3 – Optical See-Through HMD:** Comprende visori immersivi trasparenti in cui l'utente osserva l'ambiente reale direttamente con i propri occhi attraverso lenti o specchi semitrasparenti, dove il sistema ottico proietta e sovrappone gli ologrammi sintetici. In questo caso l'allineamento tridimensionale è critico e l'occlusione visiva è complessa, poiché gli ologrammi tendono a risultare traslucidi [1].

4. **Classe 4 – Video See-Through HMD:** Comprende visori immersivi a vista chiusa nei quali il mondo esterno viene digitalizzato in tempo reale da fotocamere stereoscopiche montate sul visore. In questo caso viene risolto il problema dell'occlusione della classe 3, infatti il calcolatore ha il pieno controllo sia sui pixel reali (video pass-through), sia su quelli virtuali, potendo fonderli perfettamente nello spazio virtuale [1].

5. **Classe 5 – Completely Graphic Displays with Video (Augmented Virtuality(AV)):** Sono ambienti prevalentemente sintetici e modellati in 3D, immersivi o semi-immersivi, all'interno dei quali vengono inseriti elementi o flussi video reali [1].

6. **Classe 6 – Partially Immersive Graphic Displays with Real Interaction:** Sono ambienti prevalentemente grafici (es. proiettati su grandi schermi) nei quali oggetti fisici reali appartenenti all'ambiente dell'operatore interagiscono direttamente con la scena virtuale [1].

1.2 Sistemi Virtuali e Robotici nella Riabilitazione Medica

Al giorno d'oggi, l'integrazione tra manipolatori robotici e interfacce di visualizzazione immersiva offre diverse opportunità nell'assistenza clinica, nella chirurgia guidata e nella riabilitazione medica [3]. I robot collaborativi, anche detti *cobot*, mettono a disposizione diverse caratteristiche utili all'utilizzatore tra cui un'elevata accuratezza cinematica, ripetibilità delle traiettorie compiute e la capacità di fornire forze fisiche controllate [5].
L'unione tra la robotica e la Realtà Aumentata Spaziale si basa su una collaborazione reciproca delle due tecnologie. Nel complesso, il robot garantisce flessibilità fisica e sicurezza tramite il controllo di impedenza, infatti, grazie a diversi algoritmi, la macchina non esegue movimenti del tutto rigidi, ma regola la propria flessibilità meccanica, permettendo all'operatore di interagire con il braccio e deviarne la traiettoria manualmente e in modo sicuro [6]. Dall'altro lato c'è il visore, che interviene in sostegno dell'uomo riducendone il carico cognitivo (*Cognitive Load Theory*). Nella pratica tradizionale, infatti, un operatore è costretto a leggere i dati sui monitor esterni per poi proiettarli mentalmente sulla scena reale [3]. La Realtà Aumentata Spaziale va ad eliminare questo sforzo di astrazione, mostrando i vari elementi sintetici direttamente sovrapposti al campo visivo reale, esattamente dove l'azione ha luogo [2, 3].
Nonostante i vantaggi che vengono offerti combinando queste due diverse tecnologie, esse non sono esenti da problemi strutturali e di comunicazione tra le due. Queste diverse problematiche vengono descritte nel seguente paragrafo.

1.3 Problematiche e Sfide Ingegneristiche

Tra i principali problemi che emergono nella realizzazione di sistemi a Realtà Aumentata Spaziale applicati alla robotica si ha quello della **Latenza di Rete e Latenza *Motion-to-Photon*** [7], cioè un ritardo complessivo di sistema

superiore ai 10–20 ms, che provoca errori di registrazione dinamica facendo oscillare gli ologrammi rispetto ai riferimenti fisici reali, questo effetto prende il nome di *swimming effect* [2, 7]. Un altro problema è rappresentato dal ***Motion Sickness*** e dai **Conflitti Sensoriali**, infatti, disallineamenti temporali o spaziali tra il flusso visivo del visore e la percezione dell'equilibrio dell'operatore generano senso di affaticamento visivo e nausea, fenomeno anche chiamato *cybersickness* [2, 3]. Altro aspetto fondamentale da tenere conto è **L'Accuratezza dell'Allineamento Spaziale**, infatti, l'ancoraggio geometrico millimetrico tra il sistema di riferimento del robot e la scena digitale è indispensabile per la sicurezza e la precisione dell'applicazione che si utilizza [2]. Ultimo aspetto da considerare sono i **Limiti Computazionali Dei Dispositivi Standalone**, dato che i visori richiedono l'esecuzione simultanea di tracciamento SLAM, decodifica del pass-through e rendering 3D elevato frame-rate, è pertanto necessario che, già in fase di progettazione, il carico computazionale venga gestito in maniera adeguata [8].

Definite le principali applicazioni e le problematiche della fusione tra il mondo dei dispositivi robotici e quello dei dispositivi di visualizzazione sintetica, la seguente trattazione introduce la struttura generale e le caratteristiche del progetto sviluppato a cui essa fa riferimento.

1.4 Inquadramento del Progetto: Il Digital Twin in Realtà Aumentata Spaziale

Il seguente progetto affronta le criticità sopra descritte sviluppando un'architettura di *Realtà Aumentata Spaziale* basata sul modello del **Digital Twin** in tempo reale per la telemetria del manipolatore collaborativo **KUKA LBR Med 14 R820** tramite visore **Meta Quest 3** [6, 9]. Parte fondamentale del progetto risiede nella ridefinizione del concetto di *Digital Twin* in relazione al modello di visualizzazione adottato, infatti, se il progetto fosse stato sviluppato in VR, essendo l'utente visivamente isolato dall'ambiente fisico, il sistema sarebbe costretto a renderizzare il modello CAD 3D completo dell'intera struttura meccanica del KUKA per consentire all'operatore di comprendere la posizione spaziale del robot [10]. Dato che il lavoro di tesi è stato sviluppato in AR Spaziale, il concetto di Digital Twin cambia: sfruttando il *Video-Passthrough* a colori del Meta Quest 3, l'operatore osserva direttamente il braccio meccanico reale nel proprio campo visivo [11]. Rappresentare una struttura olografica completa e sovrapporla al robot fisico rappresenterebbe uno spreco superfluo di risorse computazionali (causando quindi una penalizzazione del frame-rate di Unity) e genererebbe fenomeni di ostruzione visiva con conseguente affa-

ticamento ottico [2, 11]. Per tali ragioni, il sistema sviluppato implementa il **Digital Twin della Flangia del Robot**. In robotica, il punto operativo principale è rappresentato dalla flangia, in quanto sede dell'interfaccia di contatto e dell'applicazione delle forze [5, 12]. Nel progetto quindi, viene implementata una **Sfera Olografica Rossa** programmata per inseguire la flangia reale, che rappresenta l'essenza operativa del Digital Twin, difatti essa dimostra come il dominio fisico e quello digitale siano ancorati nel medesimo spazio metrico, garantendo la massima efficienza computazionale e un'assoluta chiarezza visiva.
L'architettura del sistema si basa su tre elementi distinti. Il controllore del manipolatore rappresenta la fonte dell'informazione e genera due flussi di natura differente: la telemetria continua, che descrive lo stato istantaneo della flangia, e l'intenzione di moto, che anticipa le traiettorie che il controllore deve eseguire nel successivo istante. Un calcolatore intermedio collega la rete del robot a quella su cui opera il visore e uniforma i dati che vengono trasmessi. L'applicazione in esecuzione sul visore riceve questo flusso di dati, ne fa una rappresentazione grafica e gestisce l'interazione con l'operatore.
Affinché avvenga la sovrapposizione fra l'ambiente virtuale e quello robotico è necessario che il sistema di riferimento del manipolatore e quello del visore siano posti in corrispondenza; tale allineamento avviene tramite l'ausilio di un marker fisico collocato alla base del manipolatore robotico. Le modalità di sviluppo di ciascuno di questi elementi sono oggetto dei capitoli successivi.

1.5 Organizzazione del Lavoro di Tesi e Risultati Conseguiti

Il lavoro descritto in questa trattazione ha portato alla realizzazione di un'applicazione funzionante, collaudata sul manipolatore, in grado di mostrare all'operatore la posizione in tempo reale della flangia del dispositivo robotico tramite un marcatore olografico posto su di essa, visualizzato in ambiente virtuale. L'allineamento fra il sistema di riferimento del robot e quello del visore viene stabilito all'inizio di ogni sessione tramite il riconoscimento di un marcatore fisico. Questa procedura rende il sistema indipendente dalla posizione del manipolatore nell'ambiente circostante, condizione necessaria data l'installazione del robot su un carrello mobile.
La trattazione prosegue, nel capitolo 2, descrivendo l'infrastruttura e gli strumenti che comprendono il lato del visore, insieme alla descrizione della comunicazione che quell'elemento ha con il resto del sistema. Nel capitolo 3 si prosegue con la presentazione della controparte robotica, illustrando l'architettura hardware del manipolatore, le librerie native usate per il controllo del moto e la struttura dei flussi dati generati dal controllore. Si passa al capitolo 4,

che costituisce il nucleo del lavoro e descrive lo sviluppo dell'applicazione, dalla procedura di ancoraggio spaziale alla conversione fra i sistemi di riferimento, fino alla realizzazione del canale di comunicazione e alla rappresentazione grafica dei dati. Il capitolo 5 tratta le prove di validazione condotte e le problematiche emerse nel corso dello sviluppo, mentre il capitolo 6 raggruppa le conclusioni ottenute e descrive le possibili evoluzioni del sistema.

Capitolo 2

Sistema di Visualizzazione e Sviluppo in Realtà Aumentata

In questo capitolo vengono analizzati gli strumenti hardware e software utilizzati per lo sviluppo del progetto, in particolare la parte inerente all'ambiente in Realtà Aumentata(AR). Il capitolo si apre con l'analisi strutturale del visore e delle sue tecnologie di acquisizione visiva inquadrate nei concetti teorici della Realtà Mista. Successivamente, viene descritto l'ambiente di sviluppo software, illustrando il motore grafico, i kit di integrazione e gli strumenti di collegamento e di testing offline utilizzati. Il capitolo si conclude con la descrizione dell'architettura di rete e le motivazioni di determinate scelte ingegneristiche che hanno portato all'adozione del protocollo UDP per la ricezione dei dati di tracciamento.

2.1 Infrastruttura Hardware: Meta Quest 3 e Tecnologia Passthrough



Figura 2.1: Visore Meta Quest 3

L'interfaccia visiva del progetto è affidata esclusivamente al visore Meta Quest 3, dispositivo autonomo di Realtà Mista che integra all'interno della sua struttura l'unità di calcolo, i display stereoscopici e l'apparato sensoriale utilizzato per il tracciamento della posizione dell'utente e delle sue mani [9]. Esso gode di un hardware avanzato; rappresenta infatti il primo visore a montare Snapdragon XR2 Gen 2 [13]. Quest'ultimo offre una GPU con un'alta potenza di calcolo, raddoppiata rispetto al suo predecessore (Meta Quest 2). Ciò permette il supporto da parte del dispositivo di applicazioni pesanti, ad esempio giochi d'azione, e soprattutto di Video Pass-through, con una certa fluidità e a colori, che consente di digitalizzare l'ambiente circostante ad alta risoluzione [13].

La Tecnologia Video Pass-through

Il pass-through è una funzione nei visori VR che consente all'utilizzatore di uscire dalla vista immersiva per visualizzare in tempo reale ciò che lo circonda [14]. Dal punto di vista elettronico, esso utilizza due fotocamere RGB da 18 PPD, protette da una piastra in vetro temperato [9], accoppiate a un proiettore di punti a infrarossi per il calcolo attivo della profondità. Volendo classificare il Quest 3 attraverso la "Tassonomia di Milgram e Kishino(1994)", esso risulta essere un sistema di tipo 4 Video See-Through (HMD Classe 4) [1], in cui l'ambiente reale viene acquisito da telecamere frontali stereoscopiche e ricostruito su display interni, con la grafica composta digitalmente sull'immagine acquisita [14, 13]. Questa modalità offre diversi vantaggi ingegneristici a livello industriale:

1. **Requisito di Mappatura Ortoscopica (1:1)**: L'appartenenza alla Classe 4 comporta per definizione che il mondo visualizzato corrisponda ortoscopicamente all'ambiente reale circostante [1, 4]: è questo requisito a rendere significativa la sovrapposizione spazialmente registrata del marker virtuale sulla flangia del robot.

2. **Risintesi del reale e compositing unificato.** A differenza dei sistemi in cui il mondo reale è osservato direttamente, nella Classe 4 gli oggetti reali sono campionati e risintetizzati su un dispositivo di visualizzazione [1]. Immagine reale e grafica sintetica convergono così nel medesimo buffer di rendering, rendendo l'occlusione fra le due un'operazione di composizione digitale anziché un vincolo ottico, vantaggio riconosciuto al video see-through rispetto all'optical see-through [2].

3. **Estensione della conoscenza del mondo (EWK).** Il sistema si colloca nella regione Where+What, disponendo sia della posizione delle superfici sia della loro identificazione semantica [1]. Per il manipolatore sono inoltre noti simultaneamente il modello geometrico e la posa istan-

tanea trasmessa dal controllore: si realizza così il caso, previsto dagli autori, in cui è il calcolatore a conoscere geometria e assetto dell'oggetto reale e a collocarvi sopra l'elemento grafico [1, 4].

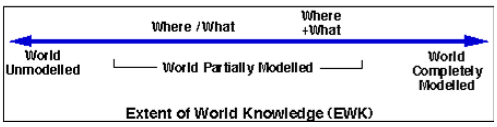


Figura 2.2: Rappresentazione dell'Estensione della Conoscenza del Mondo(EWK) secondo Milgram [1]

Un ultimo aspetto riguarda la segnalazione dello stato operativo del visore verso l'esterno. Il dispositivo integra due indicatori luminosi che comunicano il proprio stato sia all'utilizzatore sia alle persone presenti nell'ambiente [13]. Essi permettono di garantire la consapevolezza situazionale all'operatore. I LED sopra citati comprendono:

- **Status Indicator Led:** Posto sul lato destro del visore, monitora lo stato della batteria ricaricabile (Orange in ricarica tramite alimentatore da 18 W, Green a carica completa, White a dispositivo attivo) [13].



Figura 2.3: Status Indicator LED.

- **Bystander Signalling LED:** Posto sulla piastra frontale in vetro temperato, emette una luce bianca fissa quando l'apparato sensoriale del visore, formato da fotocamere monocromatiche dedicate al tracciamento della posizione del visore e delle mani e dal sensore di profondità a infrarossi, è attivo, e lampeggia in bianco quando la modalità Passthrough è attiva e l'applicazione esegue operazioni di casting o registrazione, avvisando i presenti della digitalizzazione in corso [13].



Figura 2.4: Bystander Signalling LED.

2.2 Strumenti di Sviluppo Software ed Ambiente di Emulazione

Per l'integrazione dell'ambiente a realtà aumentata, con tutte le sue caratteristiche, sono stati utilizzati: un motore grafico per la gestione dell'ambiente 3D, i kit di sviluppo per l'interfacciamento con il visore, uno strumento di collegamento PC-visore e un sistema di simulazione per lo sviluppo del visore.

2.2.1 L'Ambiente Unity e i Pacchetti Fondamentali

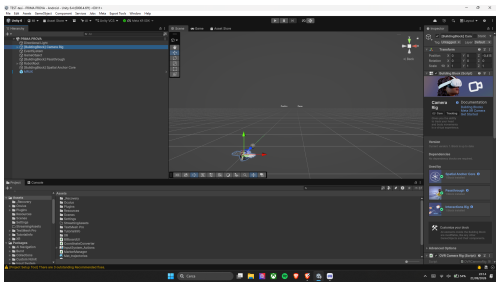


Figura 2.5: Panoramica Generale all'interno di Unity

Nel progetto, Unity è stato utilizzato come ambiente principale per la gestione della scena tridimensionale, del rendering del marker e dell'interfaccia HMI, nonché per la ricezione e l'elaborazione dei dati provenienti dal sistema robotico. Esso è un potente motore con una grande varietà di strumenti che possono essere utilizzati per soddisfare l'esigenza specifica dell'utenza [15]. L'Utilizzo di Unity è giustificato dal fatto che, oltre alla flessibilità offerta dalla programmazione in linguaggio C#, offre un'architettura orientata agli oggetti (GameObject) e componenti (Transform), che si adattano perfettamente alla manipolazione di coordinate cartesiane e vettoriali.
Per abilitare le funzionalità in realtà aumentata e ottimizzare al meglio le prestazioni del visore, il progetto si avvale di specifici pacchetti software:

- **Meta XR SDK (Core e Interaction):** È il pacchetto ufficiale di strumenti e librerie creato da Meta per permettere agli sviluppatori di realizzare applicazioni di Realta Virtuale (VR) e Realtà Mista (MR) per i visori Meta Quest [16]. Esso consente di accedere direttamente alle API del Quest 3, configurando elementi essenziali come l'`OVRCameraRig` (che sostituisce la telecamera standard di Unity per gestire la proiezione stereoscopica parallela all'occhio dell'utente) e abilitando la visualizzazione del `Video Passthrough` come sfondo della scena virtuale.

- **Unity UI (uGUI):** Utilizzato per lo sviluppo di pannelli informativi e del marker grafico tridimensionale che deve sovrapporsi alla flangia del dispositivo robotico.

2.2.2 Meta Quest Link Application

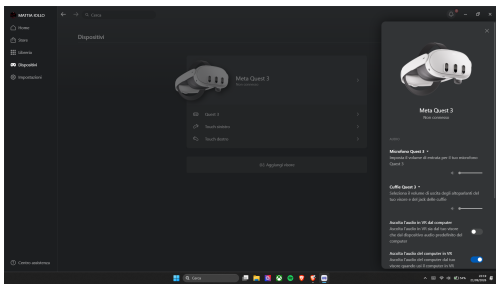


Figura 2.6: Panoramica Generale all'interno di Meta Horizon Link

Per evitare di dover effettuare i vari test installando ogni build del programma sul visore, si utilizza l'applicazione Meta Quest Link (su PC denominata Meta Horizon Link). Essa permette di eseguire l'applicazione dal computer al visore tramite collegamento ad alta banda (via USB-C o wireless con Air Link) e di avviarla direttamente dall'editor di Unity [16]. Offre inoltre la possibilità allo sviluppatore di eseguire il debug del codice C# in tempo reale e osservare istantaneamente il riscontro visivo all'interno del visore, senza dover compilare l'applicazione.

2.2.3 Meta XR Simulator ed Emulazione del Visore

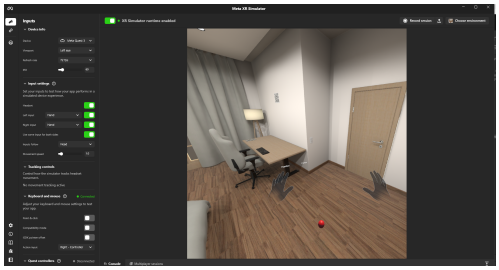


Figura 2.7: Panoramica Generale all'interno di Meta XR Simulator

Durante le fasi di sviluppo in cui non è stato possibile utilizzare l'hardware del Meta Quest 3, si è potuto procedere con l'avanzamento del progetto grazie al **Meta XR Simulator** (integrato nel Meta XR SDK).
Questo programma consente di simulare le caratteristiche e funzionalità fornite dal visore e dai suoi controller attraverso l'utilizzo di mouse e tastiera. Tra i vari vantaggi a livello ingegneristico che offre il simulatore si hanno:

- **Simulazione del Tracking e delle Telecamere:** Tramite il simulatore è possibile testare il movimento della testa dell'operatore con il visore; in questo modo è possibile verificare la stabilità di eventuali oggetti presenti nello spazio virtuale (in questo caso un marker virtuale).
- **Testing dei canali di comunicazione:** Grazie al simulatore è stato possibile testare il corretto funzionamento dello script di comunicazione UDP e dell'algoritmo di conversione delle coordinate cartesiane, inoltre, tramite diversi script in Python che generavano diversi pacchetti UDP fittizi (che simulavano l'invio dei dati da parte del KUKA) è stato possibile sfruttare il simulatore per validare l'allineamento dei sistemi di riferimento anche in totale assenza del robot reale.
- **Ottimizzazione del Flusso di Lavoro:** Una nota di particolare risiede nel fatto che, attraverso il simulatore, sono stati diminuiti i tempi di setup fisico e di affaticamento visivo/posturale legato all'uso prolungato del visore nelle fasi di test degli script nelle fasi di scrittura di codice.

2.3 Architettura di Ricezione Dati e Protocollo UDP

La trasmissione dei pacchetti dati dal dominio robotico KUKA all'interfaccia AR è affidata al protocollo `UDP (User Datagram Protocol)`, definito all'interno della specifica `RFC 768` [17]. La scelta del suddetto protocollo di comunicazione anziché del protocollo `TCP(Transmission Control Protocol)` [18]

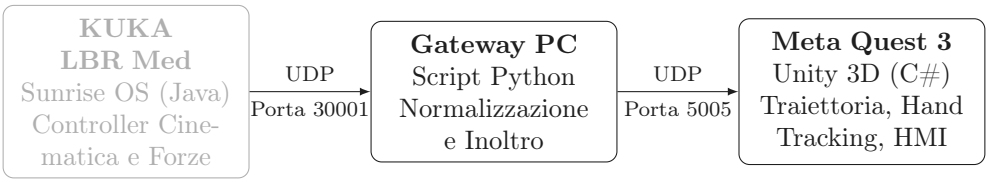


Figura 2.8: Catena di comunicazione fra il manipolatore e il visore. In evidenza il segmento trattato nel presente capitolo, relativo alla ricezione dei datagrammi da parte dell'applicazione in esecuzione sul visore.

non è stata casuale, ma è stata definita da diversi fattori ingegneristici fondamentali, come soddisfare i vincoli di sincronizzazione temporale e reattività visiva tipici della Realtà Aumentata.

2.3.1 Motivazioni Ingegneristiche per la Scelta del Protocollo: Analisi Comparativa tra UDP e TCP

La scelta di utilizzare il protocollo UDP piuttosto che il TCP è dovuto a diverse specifiche che hanno fornito diversi vantaggi durante la progettazione:

- **Priorità Temporale Contro Affidabilità di Consegna("Fire and Forget"):** Il protocollo TCP nasce per garantire che ogni pacchetto arrivi a destinazione integro ed in sequenza, questo tramite dei meccanismi automatici di riscontro, anche chiamati Arknowledgment (ACK), e di ritrasmissione in caso di perdita lungo la rete [18]. Nello streaming dei pacchetti dati ad alta frequenza (nel caso del seguente progetto impostato a 50 Hz, con un nuovo pacchetto generato dal task in background sul KUKA ogni 20 ms), le informazioni di posizione passate, ad esempio, perdono istantaneamente di valore. Supponendo di smarrire un pacchetto dati contenente la posizione numero N durante la trasmissione al visore, richiedere la sua ritrasmissione non trova alcun vantaggio; infatti, durante il tempo gestito per la ritrasmissione, il braccio robotico si sarà già spostato alla posizione N+k. L'UDP opera secondo la logica "Fire and Forget" non orientata alla connessione [17], cioè un modello di comunicazione asincrona in cui un'applicazione invia un messaggio a un'altra applicazione e prosegue immediatamente la propria esecuzione, senza attendere una risposta o una conferma di ricezione [19]. Nel caso del protocollo UDP quindi, il dato obsoleto perso viene semplicemente scartato e il visore elabora direttamente il datagramma successivo arrivato 20 ms dopo.

- **Assenza di Head of Line Blocking e Zero Stuttering:** L'adozione del protocollo TCP avrebbe esposto il sistema al problema dell' `Head-of-line Blocking`, cioè il fenomeno in cui la perdita o il ritardo

di un singolo datagramma causerebbe l'arresto della coda di ricezione dati sia nello stack di rete sia a livello del socket in Unity [20]. Tale meccanismo costringerebbe l'applicazione ad attendere la ritrasmissione del dato mancante prima di poter elaborare i pacchetti successivi. All'interno del visore Meta Quest 3, questo si traduce, a livello percettivo, come un temporaneo arresto del marker spaziale, seguito da bruschi microscatti (stuttering o jerk) dovuti all'elaborazione simultanea dei pacchetti accumulati nel buffer [8]. Al contrario, l'impiego dell'UDP evita ogni interruzione per quanto riguarda il rendering nel visore: il sistema assicura una visualizzazione della traiettoria e del marker fluida e visivamente coerente, questo grazie a un'elaborazione del flusso di dati in modo ininterrotto e soprattutto senza vincoli di consegna sequenziale [17].

- **Minimizzazione dell'Overhead e della Latenza:** A differenza del protocollo TCP, caratterizzato da un'intestazione complessa, cioè da 20 a 60 byte, e dalla necessità di connessione tramite Three-Way-Handshake [18], il protocollo UDP utilizza un header minimale di soli 8 byte, composto unicamente dai campi Source Port, Destination Port, Length e Checksum [17]. Questa riduzione dell'overhead di rete riduce il carico computazionale richiesto dalla CPU del visore, in questo modo viene minimizzata la latenza `end-to-end`, cioè il tempo che intercorre tra un'azione dell'utente e la visualizzazione del risultato aggiornato nel visore [8]. Nel contesto della Realtà Aumentata, questo miglioramento temporale è essenziale per prevenire difetti di allineamento visivo nel tracciamento continuo e per evitare l'insorgenza di disturbi visivo-motori nell'operatore [2].

2.3.2 Implementazione del Socket Ricevitore C# in Unity

Sul lato software in Unity, la ricezione dei datagrammi UDP viene gestita tramite una classe C# dedicata che inizializza un socket in ascolto sulla porta di rete configurata. Per prevenire il congelamento del ciclo di rendering 3D, l'ascolto dei pacchetti di rete viene eseguito all'interno di un thread secondario in background. I payload ricevuti, decodificati come stringhe UTF-8, sono formattati in standard JSON. Tali stringhe vengono immagazzinate in una coda thread-safe per essere poi consumate dal thread principale in Unity. Tramite le librerie di deserializzazione native, il JSON viene mappato dinamicamente su classi C# serializzabili, estraendo i valori float relativi alla telemetria: coordinate cartesiane di posizione (X, Y, Z), valori delle forze esterne (Fx, Fy, Fz) e coordinate per le traiettorie predittive. Infine i dati processati vengono impiegati per aggiornare in tempo reale la componente `Transform` del marker spaziale e i pannelli informativi dell'interfaccia HMI.

Capitolo 3

Architettura e Controllo del Sistema Robotico

In questo capitolo viene analizzata l'architettura fisica, software e di rete utilizzata per il controllo del manipolatore robotico e per la generazione della telemetria. Viene presentata la struttura del robot collaborativo KUKA LBR Med, seguita da un approfondimento dell'ambiente di sviluppo e delle API native Java impiegate per il controllo di impedenza cartesiana, la generazione del moto in tempo reale e la stima delle forze di reazione. Infine, viene descritta la gestione dei due flussi di dati (telemetria continua e intenzione di moto predittiva) e la loro serializzazione in formato JSON tramite socket UDP.

3.1 Infrastruttura Hardware: Il Manipolatore KUKA LBR Med 14 R820

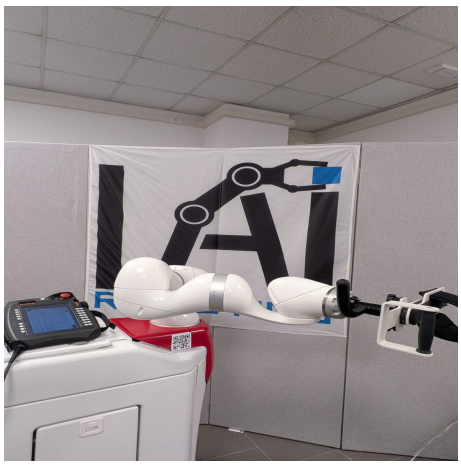


Figura 3.1: KUKA LBR Med 14 R820

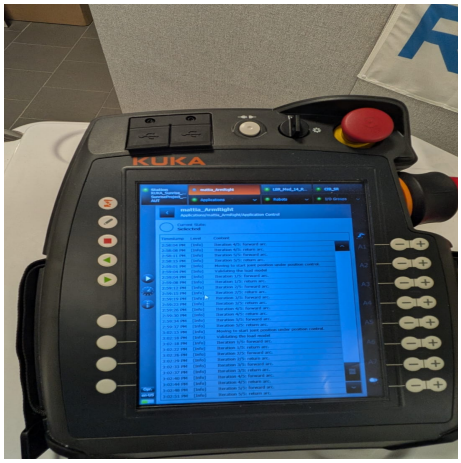


Figura 3.2: smartPAD Utilizzato dal Dispositivo Robotico

La componente robotica del sistema si basa sul manipolatore leggero collaborativo `KUKA LBR Med 14 R820`, integrato con l'unità di controllo `KUKA Sunrise Cabinet Med` [6]. Questo dispositivo è progettato per operare a stretto contatto con l'operatore in ambienti industriali e medicali ad alta sicurezza [6, 5]. Il dispositivo presenta una struttura meccanica a 7 gradi di libertà (7 Dof) che conferisce al braccio ridondanza cinematica, permettendo di variare la configurazione dei giunti interni senza alterare la posizione cartesiana del Tool Center Point(TCP), cioè il punto centrale dell'utensile (o end-effector) montato sulla flangia del robot [5, 12]. Ciascuno dei 7 giunti integra un sensore di coppia giunto dedicato (GMS-Gelenkmomentensensor) [5], che consente sia il monitoraggio attivo degli sforzi esterni sia l'abilitazione di modalità di interazione sicura [5, 12].

3.2 Ambiente di Sviluppo e Librerie Native(Sunrise.Workbench med)

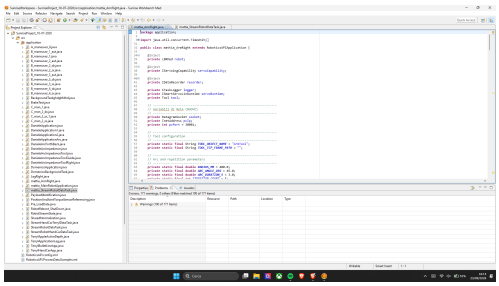


Figura 3.3: Panoramica all'interno di SunriseWorkspace

Il software di controllo all'interno del robot è sviluppato in linguaggio Java all'interno dell'ambiente di sviluppo `KUKA Sunrise.Workbench Med` [5]. La

struttura del codice fa uso delle librerie e delle API native di `KUKA Sunrise.OS Med` per la gestione avanzata del moto e della sensoristica [5], tra cui:

- **Controllo di Impedenza Cartesiana (`CartesianImpedanceControlMode`):** Per garantire la sicurezza durante la presenza dell'operatore nel raggio d'azione del robot, il controllo del braccio non si limita alla rigida posizione geometrica, ma adotta un modello dinamico ad impedenza cartesiana [5]. Tramite la classe `CartesianImpedanceControlMode` è possibile definire dei parametri di rigidezza e smorzamento variabili lungo i 3 assi cartesiani (X, Y, Z) e le rispettive rotazioni (A, B, C) [5]. Questo rende il robot meccanicamente sicuro e collaborativo, consentendo all'operatore di applicare spinte fisiche dirette e deviare temporaneamente la traiettoria in totale sicurezza.

- **Generazione di Moto Ciclico Real-Time (`ISmartServoLin`):** La pianificazione e l'aggiornamento dinamico della traiettoria sono affidati al pacchetto `KUKA Sunrise.Servoing.Med` tramite l'interfaccia ISmartServoLin [5]. A differenza delle istruzioni di moto tradizionali di natura bloccante, `ISmartServoLin` permette di ricalcolare la traiettoria in tempo reale. Il sistema scarta istantaneamente le coordinate di arrivo obsolete, inseguendo in modo fluido i nuovi punti di passaggio generati ciclicamente dal controllore a bassissima latenza [5].

- **Estrazione delle Forze Cartesiane (`ForceData` e `ForceTorqueData`):** L'acquisizione delle interazioni meccaniche sulla flangia del robot avviene attraverso i sensori di coppia presenti nei giunti del manipolatore [5]. Le classi native `ForceData` e `ForceTorqueData` estraggono i vettori in tempo reale espressi in newton (N) lungo le componenti cartesiane F_x, F_y, F_z [5]. Come convenzione fisica, le forze stimate presentano segno opposto rispetto alla direzione di avanzamento del robot, rappresentando la reazione opposta dell'ambiente o dell'operatore.

3.3 Pianificazione della Traiettoria e Controllo del Moto

Nella teoria della robotica viene fatta la distinzione fra **cammino** e **traiettoria** [21]. Il cammino viene definito come il luogo geometrico dei punti che la flangia del robot deve attraversare nello spazio operativo ed è quindi una curva priva di qualsiasi riferimento temporale; la traiettoria invece è il cammino al quale è stata associata una legge oraria, cioè una funzione che stabilisce in quale istante ciascun punto debba essere raggiunto e che determina di conseguenza i valori di velocità e di accelerazione [21]. La pianificazione consiste nel

definire una traiettoria a partire da pochi dati forniti dall'utente, come i punti da raggiungere e i vincoli delle grandezze cinematiche. Da questi dati viene poi generata una sequenza di riferimenti che il controllore utilizza per guidare il movimento del sistema.
L'ambiente Sunrise mette a disposizione due famiglie di primitive di moto, cioè il moto punto-punto e il moto continuo [5]. Il moto punto-punto viene pianificato direttamente nello spazio dei giunti: il controllore interpola le posizioni angolari fra configurazione iniziale e finale, ottenendo il tempo di percorrenza minimo ma lasciando indeterminato il cammino descritto dalla flangia nello spazio cartesiano. Il moto continuo viene invece pianificato nello spazio operativo e impone alla flangia un cammino geometricamente assegnato, un segmento lineare o un arco di circonferenza. L'istruzione circolare richiede, oltre a un punto di arrivo, un punto ausiliario intermedio: tre punti non allineati individuano una sola circonferenza nello spazio tridimensionale, e la posizione del punto ausiliario stabilisce quale dei due archi delimitati dagli estremi debba essere percorso e in quale verso. Per questo motivo i messaggi di intenzione di moto descritti nel paragrafo 3.4 trasportano, nel caso circolare, tre terne di coordinate invece di due, e la ricostruzione grafica realizzata in ambiente Unity, discussa nel paragrafo 4.6, deve replicare lo stesso criterio di selezione.

Insieme alle istruzioni di moto tradizionali, che sono di tipo bloccante e restituisce il controllo al programma solo al termine dell'esecuzione, il pacchetto `Sunrise.Servoing.Med` consente la generazione del moto in tempo reale attraverso l'interfaccia `ISmartServoLin` [5]. In questa modalità l'applicazione non assegna un unico punto di arrivo, ma aggiorna ciclicamente la destinazione corrente con il periodo del lavoro da effettuare, pari a 20 ms, mentre il controllore si occupa dell'interpolazione fra un riferimento e il successivo. Il percorso complessivo viene quindi suddiviso in diversi piccoli spostamenti consecutivi, e il modo con cui questi vengono distribuiti nel tempo determina la fluidità del movimento percepita dall'osservatore. Durante la progettazione è stata adottata una legge oraria dove il braccio parte da fermo, accelera progressivamente fino a raggiungere la velocità massima, finché non rallenta in modo simmetrico fino ad arrestarsi al punto di arrivo. In questo modo vengono evitate le partenze e gli arresti bruschi che si otterrebbero assegnando una velocità costante lungo tutto il percorso. Questa caratteristica ha notevole importanza in un contesto collaborativo, dove movimenti improvvisi risulterebbero potenzialmente pericolosi per l'operatore che si trova nelle vicinanze del dispositivo robotico.

Per quanto riguarda il livello di **controllo**, esso viene definito come il modo in cui il riferimento pianificato viene effettivamente inseguito dagli attuatori.

Un controllo di posizione puro insegue il riferimento in maniera rigida e interpreta qualunque scostamento come un errore da annullare. Questo tipo di comportamento risulta essere inadatto a un manipolatore che deve interagire con una persona [21]. Il controllo di impedenza cartesiana che adotta il progetto segue una logica diversa, infatti al braccio viene attribuito un comportamento cedevole. Se l'operatore spinge la flangia, questa si allontana dal riferimento in misura proporzionale alla forza applicata, e vi torna gradualmente quando la spinta cessa. La rigidezza impostata per ciascuna direzione stabilisce quanto il robot si opponga a questo allontanamento: con una rigidezza elevata il braccio insegue il riferimento in modo quasi rigido, mentre una rigidezza ridotta consente deviazioni notevoli anche con spinte modeste. Da queste affermazioni è possibile apprendere che la posizione effettivamente occupata dalla flangia non coincide necessariamente con quella pianificata, e che la differenza fra le due non rappresenta una problematica del sistema, ma la dimostrazione di quanto detto precedentemente riguardo al comportamento cedevole del manipolatore.

3.4 Gestione dei Flussi Dati di Telemetria e Intenzione di Moto

L'applicazione presente sul controller KUKA, è stata progettata per gestire e trasmettere all'esterno due flussi di dati diversi fra loro:

- **Telemetria Continua (Stato Istantaneo):** Un task di background ciclico esegue un campionamento deterministico a 50 Hz (un pacchetto ogni 20 ms). Ogni datagramma di telemetria contiene la posizione cartesiana istantanea della flangia del robot (X, Y, Z) e le componenti del vettore forza (F_x, F_y, F_z) rilevate sulla flangia.
- **Intenzione di Moto (Previsione della Traiettoria):** Oltre allo stato istantaneo, il software calcola in anticipo i riferimenti geometrico-spaziali della traiettoria futura prima che i motori eseguano il movimento fisico effettivo. Vengono infatti determinati ed esportati i punti di passaggio chiave (ad esempio punto iniziale, ausiliario e di arrivo per un arco, oppure punto iniziale e finale per un segmento lineare). Questo flusso di dati predittivo consente all'interfaccia di Realtà Aumentata in Unity di tracciare in anticipo la linea guida della traiettoria futura, ottimizzando la percezione e il controllo dell'ambiente di lavoro da parte dell'utente.

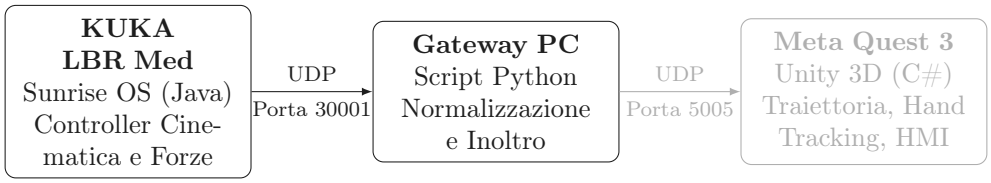


Figura 3.4: Catena di comunicazione fra il manipolatore e il visore. In evidenza il segmento trattato nel presente capitolo, relativo alla generazione dei flussi dati e alla loro trasmissione da parte del controllore.

3.5 Trasmissione della Telemetria e Serializzazione del Payload JSON

Per la trasmissione dati verso la rete locale e il successivo gateway di elaborazione, il controller KUKA opera come un server di dati in streaming. La comunicazione di rete è realizzata mediante le librerie standard Java (java.net.DatagramSocket). Sia i dati di telemetria istantanea sia i nodi della traiettoria predittiva vengono serializzati in stringhe strutturate secondo lo standard JSON.

3.5.1 Vantaggi del Formato JSON su Socket UDP

- **Strutturazione Flessibile ed Estensibile:** L'organizzazione del JSON consente di unificare sia variabili scalari (*timestamp*, contatore pacchetti) sia array vettoriali (le componenti di posizione cartesiana X, Y, Z, i punti di traiettoria e le forze F_x, F_y, F_z) in un unico pacchetto autodescrittivo. Questa scelta architetturale rende il carico utile facilmente estensibile in futuro e immediatamente decodificabile dallo script Python di intermediazione e da Unity.

- **Efficienza del Protocollo UDP(Porta 30001):** I pacchetti JSON vengono trasmessi via UDP sulla porta utente 30001. L'assenza delle sovrastrutture di controllo del TCP (come il *Three-Way Handshake* o la ritrasmissione dei pacchetti persi) azzera i ritardi di accodamento (*Head-of-Line Blocking*) [18, 20], garantendo che il sistema di visualizzazione AR riceva i dati di telemetria e le traiettorie predittive con latenze minime e senza ritardi di accodamento introdotti dal protocollo.

Capitolo 4

Progettazione e Sviluppo dell'Applicazione in Realtà Aumentata

4.1 Introduzione al Progetto

Nei capitoli precedenti sono state descritte le basi teoriche della Realtà Mista, le caratteristiche hardware e software della parte del progetto relativa all'ambiente virtuale e al manipolatore KUKA LBR med 14 R820, definendo l'architettura generale per la costruzione di un Digital Twin in real time. Questo capitolo va a descrivere nel dettaglio la fase sperimentale e lo sviluppo software dell'applicazione in Realtà Aumentata (AR), parte centrale del lavoro di tesi.
L'obiettivo principale di questa fase progettuale è stato quello di gestire il flusso di dati grezzi in uscita dal sistema robotico (telemetria e forze) in un'unica interfaccia olografica fluida, interattiva e geometricamente allineata al mondo fisico. Il capitolo è strutturato secondo l'ordine cronologico di sviluppo: dopo un richiamo degli strumenti analitici sulle trasformazioni rigide, si passa alla calibrazione spaziale utilizzando un marker fisico (QR CODE) per l'allineamento dei sistemi di riferimento cartesiani tra il mondo reale e l'ambiente virtuale di Unity. Successivamente viene affrontato il problema matematico della rototraslazione delle coordinate, necessario per far comunicare coerentemente lo spazio di lavoro del KUKA con il software unity e, successivamente, il motore grafico del visore. Nella seconda parte del capitolo viene trattata la parte riguardante la struttura software, descrivendo l'implementazione del client UDP per la ricezione asincrona dei payload JSON e la successiva decodifica dei dati. Infine, viene descritto il processo di rendering visivo della flangia del robot, delle traiettorie predittive e della Human-Machine Interface (HMI), gestita nativamente tramite il tracciamento delle mani dell'operatore (*Hand*

Tracking).

4.2 Richiami sulle Trasformazioni Rigide

Nei paragrafi successivi i dati del manipolatore vengono trasformati più volte prima di essere visualizzati nel visore. Per rendere più chiari questi passaggi si richiamano qui, in forma sintetica, i concetti di base sulle rotazioni e sulle rototraslazioni, secondo il testo *Modelling and control of Robot Manipulators* di Siciliano e Sciavicco [21].

Una terna di riferimento è formata da tre assi perpendicolari fra loro. L'orientamento di una terna 1 rispetto a una terna 0 è descritto da una matrice di rotazione $\mathbf{R}$ di dimensione 3×3, le cui colonne contengono le direzioni dei tre assi della terna 1 viste dalla terna 0. Poiché gli assi sono perpendicolari e di lunghezza unitaria, la matrice gode di una proprietà molto utile: per invertirla basta trasporla,

$$\mathbf{R}^{-1} = \mathbf{R}^{T}. \tag{4.1}$$

Se le due terne hanno anche origini diverse, alla rotazione si aggiunge uno spostamento $\mathbf{o}$, e un punto si trasforma come $\mathbf{p}^{0} = \mathbf{R}\,\mathbf{p}^{1} + \mathbf{o}$. Questa combinazione di rotazione e traslazione è la rototraslazione, che si scrive in un'unica matrice 4×4 detta trasformazione omogenea:

$$\mathbf{A} = \begin{bmatrix} \mathbf{R} & \mathbf{o} \\ \mathbf{0}^{T} & 1 \end{bmatrix}. \tag{4.2}$$

Più rototraslazioni in sequenza si combinano moltiplicando le rispettive matrici, mentre l'inversa si ottiene trasponendo la rotazione e ruotando lo spostamento cambiato di segno:

$$\mathbf{A}^{-1} = \begin{bmatrix} \mathbf{R}^{T} & -\mathbf{R}^{T}\mathbf{o} \\ \mathbf{0}^{T} & 1 \end{bmatrix}. \tag{4.3}$$

Il determinante di una matrice di rotazione può valere soltanto $+1$ oppure -1, e il suo segno ha un significato geometrico preciso. Con $+1$ la matrice è una rotazione vera e propria: sposta la terna ma la lascia del tipo che era, destrorsa o sinistrorsa. Con -1 la matrice contiene una riflessione, cioè l'equivalente di osservare la terna allo specchio, e una terna destrorsa diventa sinistrorsa. L'esempio più semplice è l'inversione di un solo asse:

$$\mathbf{D}_{x} = \operatorname{diag}(-1,\ 1,\ 1), \qquad \det \mathbf{D}_{x} = -1. \tag{4.4}$$

Poiché il determinante di un prodotto è il prodotto dei determinanti, una sequenza di rotazioni ha sempre determinante $+1$: per questo nessuna rotazione,

per quanto complessa, può cambiare il verso di una terna [21, 22].

Infine, per descrivere gli orientamenti Unity non utilizza direttamente le matrici ma i quaternioni unitari. Una rotazione di un angolo ϑ attorno a un asse di direzione $\mathbf{r}$ è rappresentata dalla coppia

$$q = \{\eta,\ \boldsymbol{\epsilon}\} = \{\cos(\vartheta/2),\ \sin(\vartheta/2)\,\mathbf{r}\}. \tag{4.5}$$

Rispetto agli angoli di Eulero i quaternioni non presentano singolarità, e le operazioni restano le stesse delle matrici: il prodotto di due quaternioni equivale a eseguire una rotazione dopo l'altra, mentre il coniugato $q^{-1} = \{\eta,\ -\boldsymbol{\epsilon}\}$ rappresenta la rotazione inversa [21].

Questi strumenti vengono applicati nei paragrafi successivi: la media degli orientamenti del marker viene calcolata sui quaternioni, la conversione fra il riferimento del KUKA e quello di Unity è una matrice con determinante -1, e la collocazione del `RobotRoot` è una rototraslazione, ottenuta componendo la posa misurata del marker con l'inversa del suo scostamento dalla base.

4.3 Ancoraggio Spaziale e Calibrazione tramite Marker QR CODE

Il primo punto trattato durante lo sviluppo dell'applicazione è stato il problema dell'allineamento dei sistemi di riferimento. Per far in modo che la sovrapposizione olografica sulla flangia del robot sia coerente, si necessita che il sistema di riferimento dell'ambiente virtuale all'interno del motore Unity e l'origine del sistema di riferimento del manipolatore KUKA coincidano nello spazio metrico. Per questo motivo si è scelto di utilizzare un metodo di tracciamento ottico basato su un Marker, nello specifico un QR CODE bidimensionale, posizionato alla base del robot. Per l'implementazione del sistema di riconoscimento, si è fatto ausilio delle API native fornite dal pacchetto **Mixed Reality Utility Kit (MRUK)**. Queste librerie, rispetto ai classici sistemi di image-tracking, permettono di incaricare il calcolo della *Computer vision* direttamente all'hardware specializzato del visore, restituendo così la posizione spaziale del marker [23]. Il sistema in questione è stato gestito tramite lo script C# `MarkerManager.cs`. Questo codice funge da filtro dei dati in ingresso, accettando la calibrazione tramite QR CODE solo se esso contiene una stringa impostata dall'operatore, in questo caso `KUKA_BASE`, in modo tale da evitare scansioni di QR CODE errati.

Per quanto riguarda le problematiche riscontrate durante le prime fasi di test, si è andati incontro a situazioni di instabilità del sensore ottico, dovute al rumore video e alle variazioni di illuminazione che andavano quindi a causare oscillazioni della posizione del marker stimata dal visore, fenomeno chiamato

Jittering. Per moderare questo fenomeno e garantire una sincronizzazione al massimo della precisione con l'origine del KUKA, è stato implementato un algoritmo di campionamento che acquisisce la posizione e la rotazione del marker su più frame, calcolandone la media. Nell'ambito della robotica, mentre la media delle posizioni è una semplice interpolazione vettoriale, la media delle rotazioni risulta essere un'operazione più complessa se effettuata attraverso il metodo degli angoli di Eulero, questo a causa di singolarità matematiche come il **Blocco Cardanico**, o Gimbal Lock, cioè la perdita di un grado di libertà in un sistema di rotazione tridimensionale quando due assi si allineano e diventano paralleli [21]. Quindi, secondo la teoria della cinematica spaziale, il sistema software gestisce l'orientamento usufruendo della rappresentazione tramite **Quaternioni**, richiamata nel paragrafo 4.2 [21].

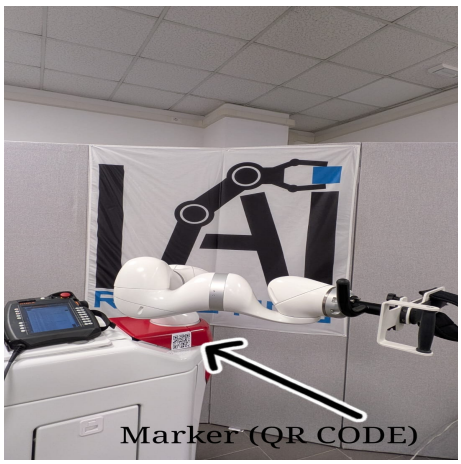


Figura 4.1: QR CODE posizionato alla base del KUKA

Viene adesso riportato lo pseudocodice del metodo `RoutineRaccolta`, che si occupa dell'estrazione dei campioni e del controllo stabilità (Algoritmo 4.1), seguito da quello delle funzioni che esso richiama (Algoritmo 4.2).

Algoritmo 4.1 Raccolta dei campioni del marker (`MarkerManager.cs`).

Ingresso: marker con contenuto `KUKA_BASE`; campioni richiesti N; soglia di stabilità s; tempo massimo T

Uscita: posa del marker mediata su N fotogrammi stabili

procedura RoutineRaccolta(marker)
    $P \leftarrow \emptyset,\ Q \leftarrow \emptyset,\ t \leftarrow 0$
    ▷ *liste di posizioni e orientamenti vuote, cronometro azzerato*
    finché P contiene meno di N campioni **esegui**
        se il marker non è più rilevato **allora**
            segnala l'errore all'operatore e **termina**
        $\mathbf{p} \leftarrow$ posizione del marker nel fotogramma corrente
        $q \leftarrow$ orientamento del marker, come quaternione
        ▷ *un salto rispetto al campione precedente rende inaffidabile*
        ▷ *quanto raccolto finora: la raccolta riparte da capo*
        se $P \neq \emptyset$ **e** $\|\mathbf{p} - \text{ultimo}(P)\| > s$ **allora**
            svuota P e Q
        aggiungi $\mathbf{p}$ a P e q a Q
        $t \leftarrow t + \Delta t$
        se $t > T$ **allora**
            segnala il timeout all'operatore e **termina**
        attendi il fotogramma successivo
    $\bar{\mathbf{p}} \leftarrow$ MediaPosizioni(P)
    $\bar{q} \leftarrow$ MediaRotazioni(Q)
    ApplicaAllineamento($\bar{\mathbf{p}}, \bar{q}$)

Algoritmo 4.2 Funzioni di media e di allineamento (`MarkerManager.cs`).

funzione MediaPosizioni(P)
    restituisci $\frac{1}{N}\sum_i \mathbf{p}_i$
    ▷ *media aritmetica, componente per componente*

funzione MediaRotazioni(Q)
    $q_{\text{rif}} \leftarrow$ primo elemento di Q, $\mathbf{m} \leftarrow (0,0,0,0)$
    per ogni $q_i \in Q$ **esegui**
        ▷ *q e $-q$ sono la stessa rotazione: si allineano i segni*
        se $q_i \cdot q_{\text{rif}} < 0$ **allora**
            $q_i \leftarrow -q_i$
        $\mathbf{m} \leftarrow \mathbf{m} + q_i$
        ▷ *somma delle quattro componenti*
    restituisci $\mathbf{m}/\|\mathbf{m}\|$
    ▷ *normalizzazione: si torna a un quaternione unitario*

procedura ApplicaAllineamento($\bar{\mathbf{p}}, \bar{q}$)
    ▷ *composizione con lo scostamento $(\mathbf{p}_O, q_O)$ fra marker e base*
    $q_R \leftarrow \bar{q} * q_O^{-1}$
    $\mathbf{p}_R \leftarrow \bar{\mathbf{p}} - q_R\,\mathbf{p}_O$
    se la base stimata è troppo lontana dall'operatore **allora**
        rifiuta la calibrazione e **termina**
    assegna $(\mathbf{p}_R, q_R)$ al `RobotRoot`

Come evidenziato dall'Algoritmo 4.1, esso controlla di continuo quanto si sposta la posizione rilevata tra un campionamento e il successivo. Se questa distanza supera una soglia prefissata (`sogliaStabilita`), vuol dire che il tracciamento non è più affidabile, allora, in questo caso i buffer, compreso quello dei quaternioni, vengono svuotati e la raccolta dei dati ricomincia da capo. Solo quando è stato accumulato un numero sufficiente di frame stabili il sistema calcola il quaternione medio e richiama la funzione `ApplicaAllineamento()`. Quest'ultima sposta e ruota l'origine degli assi (*RobotRoot*) di Unity fino a farla coincidere con la posizione misurata, così che la radice del Digital Twin resti fissata all'origine reale del KUKA.
Le tre funzioni richiamate al termine della raccolta svolgono compiti distinti. MediaPosizioni calcola la media aritmetica delle posizioni, operazione lecita perché le posizioni sono vettori ordinari. Per gli orientamenti la media aritmetica dei parametri non è invece significativa: con gli angoli di Eulero, per esempio, due misure di $179°$ e $-179°$, distanti fra loro appena $2°$, avrebbero media nulla, cioè un orientamento opposto a entrambe. MediaRotazioni opera

quindi sui quaternioni, sfruttando una loro proprietà: q e $-q$ rappresentano la stessa rotazione, poiché aggiungere un giro completo all'angolo ϑ cambia il segno di tutte e quattro le componenti senza cambiare l'orientamento. Se i campioni venissero sommati senza precauzioni, due di essi uguali ma di segno opposto si annullerebbero; per questo ogni campione il cui prodotto scalare con il primo risulta negativo viene prima cambiato di segno. Le componenti vengono poi sommate e il risultato normalizzato, in modo da ottenere di nuovo un quaternione di modulo unitario. Questa media è un'approssimazione, valida quando i campioni sono vicini fra loro [24]: condizione garantita in questo caso, perché il marker è fermo e la soglia di stabilità scarta la raccolta al primo salto anomalo. Infine, ApplicaAllineamento compone la posa media con lo scostamento fra marker e base del robot, secondo le relazioni ricavate nel Paragrafo 4.4, e prima di assegnarla al `RobotRoot` verifica che la base stimata non risulti a una distanza dall'operatore palesemente incompatibile con la scena.

4.4 Sistemi di Riferimento e Rototraslazione

Il sistema di ancoraggio spaziale descritto nel paragrafo precedente definisce dove posizionare l'origine del sistema di riferimento del Digital Twin all'interno dell'ambiente del visore, ma non è ancora sufficiente a garantire che un punto trasmesso dal controllore venga disegnato nella posizione corretta. I due sistemi, infatti, non si limitano ad avere una diversa collocazione dell'origine, ma adottano anche due convenzioni geometriche diverse che non riguardano solo l'orientamento degli assi ma la natura stessa della terna di riferimento.
Il manipolatore KUKA esprime le coordinate cartesiane rispetto al *Robot Base Coordinate System*, esse rappresentano una terna **destrorsa** posizionata alla base fisica del robot, dove l'asse Z è rivolto verticalmente verso l'alto nella configurazione di installazione [5]. Il motore grafico Unity invece utilizza una terna **sinistrorsa** in cui l'asse verticale è Y, l'asse X è diretto verso destra e l'asse Z verso la profondità della scena [25]. In questo caso stabilire quale asse rappresenti la verticale è una semplice questione di riassegnazione degli assi, mentre la differenza di chiralità, cioè il verso secondo cui la terna si avvolge sé stessa, costituisce il problema più complesso in fase di progettazione.
Per portare un punto dal riferimento del robot a quello di Unity viene utilizzata una trasformazione lineare. Essa può essere rappresentata tramite una matrice S tale che $\mathbf{p}_U = S\,\mathbf{p}_K$, dove p rappresenta un punto nello spazio e i pedici U e K stanno ad indicare rispettivamente Unity e KUKA. L'elemento caratterizzante della matrice S è rappresentato dal segno del suo determinante, infatti, nella teoria, le rotazioni proprie, che rappresentano l'oggetto comune nella cinematica dei manipolatori [21], possiedono determinante pari a $+1$ e

per questo motivo mantengono la chiralità del riferimento, infatti nessuna rotazione, per quanto complessa, può trasformare una terna destrorsa in una sinistrorsa [21]. Il passaggio richiede quindi una trasformazione ortogonale impropria con $det = -1$, cioè una riflessione [22].
Nel sistema sviluppato, la mappatura cartesiana applicata allo stato della flangia del robot associa la coordinata Z del robot alla coordinata Y di Unity, invertendo le polarità degli assi orizzontali per mantenere la coerenza della direzione di avanzamento sul piano di lavoro:

$$\begin{bmatrix} x_U \\ y_U \\ z_U \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} x_K \\ y_K \\ z_K \end{bmatrix}, \qquad \det S = -1. \tag{4.6}$$

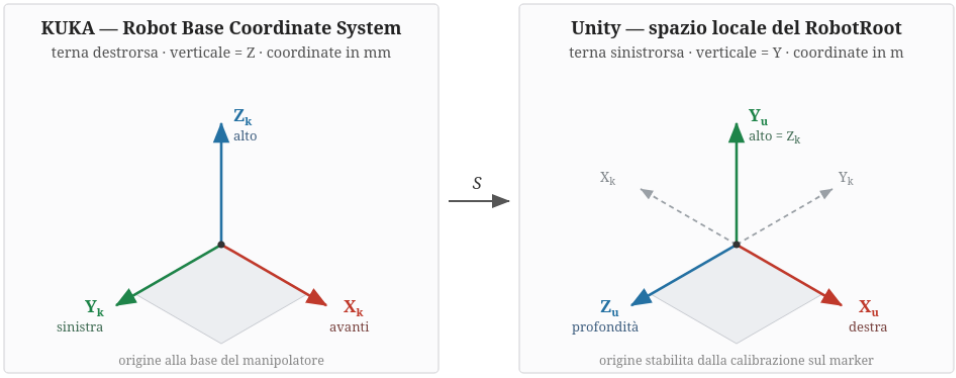


Figura 4.2: Rappresentazione del Cambio di Coordinate dal Dominio Robotico all'Ambiente Unity

Tale operazione garantisce sia il vincolo sul determinante ($detS = -1$), fondamentale per il cambio di chiralità, sia l'orientamento conforme allo spazio operativo. Questa implementazione è raccolta all'interno della classe statica `CoordinateConverter.cs`, strutturata come singolo punto di passaggio tra il dominio cinematico del robot e quello grafico del motore Unity, il cui unico metodo, ConvertKukaToUnity, applica la (4.6) componente per componente: la coordinata verticale del robot diventa quella di Unity, mentre le due componenti orizzontali vengono riassegnate con il segno invertito.

È importante precisare che i valori numerici passati in ingresso al metodo `ConvertKukaToUnity` vengono convertiti in metri prima della conversione geometrica, assicurando la piena coerenza metrica con lo spazio tridimensionale di Unity.

La conversione appena descritta risolve però soltanto una parte del problema, poiché stabilisce come orientare gli assi ma non dove collocarli: un punto

convertito risulta espresso in coordinate coerenti con Unity, ma ancora relative alla base del manipolatore. La struttura utilizzata mantiene volontariamente separati i due aspetti. Nella scena di Unity è presente un oggetto vuoto, denominato `RobotRoot`, il cui *transform* descrive la posizione della base del robot nel mondo tracciato dal visore. Gli elementi che devono coincidere con punti fisici del manipolatore, ossia la sfera olografica della flangia e le traiettorie predittive, sono collocati al di sotto di `RobotRoot` attraverso un ulteriore nodo intermedio e ricevono coordinate locali, cioè riferite ad esso; il pannello HMI, che non rappresenta un punto materiale ma deve soltanto restare solidale alla macchina, è vincolato direttamente al `RobotRoot`. In questo modo la conversione di riferimento resta una funzione fissa, scritta una sola volta, mentre l'intera variazione dovuta alla posizione fisica del robot nell'ambiente si focalizza nella posizione di un unico oggetto.
Questa posizione viene determinata al riconoscimento del marker, secondo il procedimento di campionamento descritto nel paragrafo precedente. Indicando con p_M e q_M la posizione e l'orientamento del QR code misurati dal visore, e con p_O e q_O la posizione del marker rispetto alla base del robot, la posizione del `RobotRoot` si ottiene componendo la prima con l'inversa della seconda:

$$q_R = q_M \, q_O^{-1}, \tag{4.7}$$

$$p_R = p_M - q_R \, p_O. \tag{4.8}$$

A differenza della conversione discussa in precedenza, si tratta in questo caso di una **rototraslazione** vera e propria nel senso classico del termine, cioè una rotazione a determinante positivo seguita da una traslazione, che colloca e orienta un corpo rigido senza alterarne la chiralità [21]. Le due trasformazioni che il sistema compie possiedono dunque natura matematicamente distinta, dove, la prima è nota a priori ed è costante, la seconda viene stimata nuovamente a ogni sessione di lavoro.

Il termine (p_O, q_O) esiste per una motivazione di carattere fisico. Il marker non può essere collocato in corrispondenza dell'origine del sistema di riferimento del robot, essendo essa un punto geometrico interno alla struttura della base, e si trova quindi a una certa distanza da essa. Nel sistema sviluppato questo scostamento è stato determinato sperimentalmente per misurazione diretta sul robot, infatti sono state effettuate delle misure approssimative per calcolare la distanza del marker fisico dal *Robot Base Coordinate System* con l'aggiunta di un confronto visivo con la posizione della sfera olografica con quella reale della flangia, applicando la correzione mediante un nodo intermedio nella gerarchia di scena posizionato tra il `RobotRoot` e gli elementi che inseguono punti fisici del manipolatore. Questo approccio garantisce un riscontro visivo immediato

durante la fase di sviluppo, risultando equivalente all'inserimento diretto delle coordinate di offset nelle equazioni di composizione della posizione. Questo risulta vero quando viene montato il marker con gli assi paralleli a quelli della base, in questo modo, q_O si riduce all'identità e la composizione si semplifica alla sola traslazione.

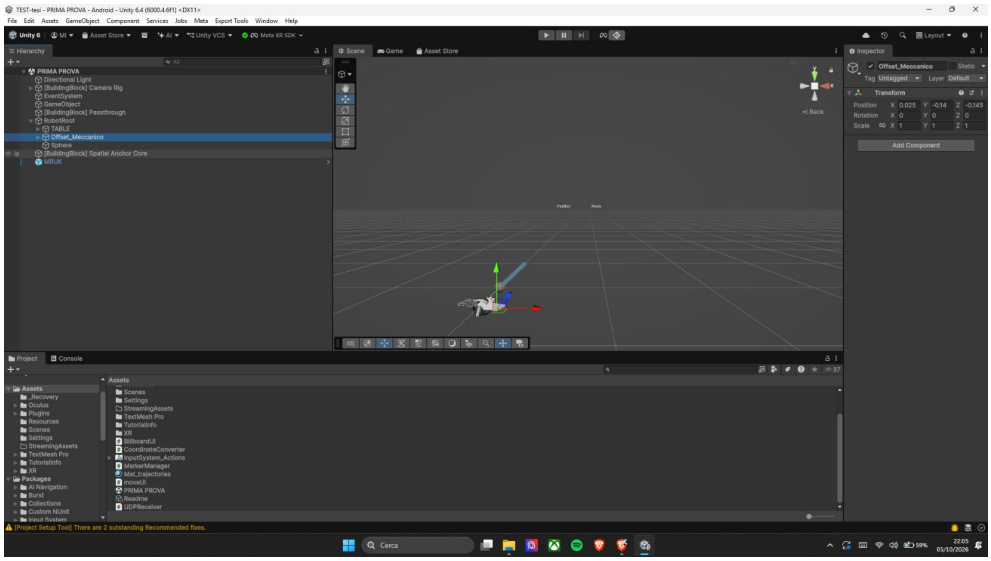


Figura 4.3: Visuale all'interno di Unity nella sezione del RobotRoot

4.5 Implementazione del Gateway Python e del Ricevitore UDP

La realizzazione software dell'architettura di comunicazione definita nella figura 2.8 viene descritta in questa sezione. Una volta definita la trasformazione geometrica delle coordinate, la struttura del progetto deve assicurare un flusso continuo, a bassa latenza e privo di ritrasmissioni per i dati generati dal manipolatore robotico: quest'ultimo produce campioni di telemetria a una frequenza nominale di 50 Hz, cioè 20 ms, a fronte di un ciclo di rendering del visore Meta Quest 3 compreso tra 72 e 90 fps. La gestione software di questa catena si divide in due componenti principali: uno script sul calcolatore (*Gateway*) che gestisce l'instradamento dei dati e la loro normalizzazione geometrica e un ricevitore in Unity strutturato per acquisire i messaggi senza interferire con il ciclo di rendering principale.

Per quanto riguarda il gateway, la necessità di un nodo intermedio deriva dall'infrastruttura generale del progetto:l'unità di controllo KUKA Sunrise Cabinet Med possiede un'interfaccia di rete Ethernet chiamata **KLI** (*KUKA Line Interface*), accessibile dall'esterno mediante il connettore fisico dedicato **X66**

[5]. Tale canale punto a punto è utilizzato per la sincronizzazione dei task Java tramite l'ambiente KUKA Sunrise.Workbench Med e per lo scambio dati telematici tramite socket UDP. La scheda di rete Ethernet del calcolatore, connessa alla porta X66, richiede per specifica del costruttore l'assegnazione di un **indirizzo IP statico** appartenente alla medesima sottorete della KLI [5]. L'indirizzamento fisso esclude l'uso del protocollo DHCP: con indirizzi dinamici, a ogni variazione o riavvio dell'interfaccia, lo streaming UDP si interromperebbe e il controllore rifiuterebbe la comunicazione. Il calcolatore è collegato alla rete locale del laboratorio mediante una seconda interfaccia di rete, condividendo la stessa rete con il visore Meta Quest 3 via wireless. Lo script `Ricevitore_adattato_traiettoria.py`, in esecuzione su una stazione pc opera come ponte tra i due elementi, riceve i datagrammi inviati dal controllore verso la porta locale 30001 e ne gestisce il rinvio verso la porta 5005 del visore.

Oltre all'indirizzamento dei dati tra le due interfacce, il compito fondamentale del gateway è la normalizzazione metrica dei dati. Il controllore robotico, infatti, esprime le coordinate cartesiane in millimetri, mentre l'ambiente virtuale di Unity usa per convenzione i metri. Effettuare questa conversione a monte del calcolatore ha una precisa motivazione: l'applicazione sul visore riceve dati già espressi nell'unità di misura nativa della scena, indipendentemente dalla sorgente. Questa separazione ha consentito, durante lo sviluppo, di alternare il manipolatore reale con script di simulazione senza dover modificare alcuna riga di codice in C# presente sul visore.

Algoritmo 4.3 Ciclo principale del gateway Python.

Ingresso: messaggi JSON del controllore (porta 30001, coordinate in mm)
Uscita: stessi messaggi inoltrati al visore (porta 5005, coordinate in m)

ripeti indefinitamente
    $m \leftarrow$ messaggio ricevuto sulla porta 30001, decodificato dal JSON
    ▷ *la conversione dipende dal tipo di messaggio; ogni coordinata*
    ▷ *viene divisa per 1000, passando dai millimetri ai metri*
    se m è un messaggio di telemetria **allora**
        converti in metri la posizione attuale della flangia
    altrimenti se m è un messaggio di previsione **allora**
        converti in metri i punti di partenza, ausiliario e di arrivo
    ▷ *le forze restano invariate, essendo già espresse in newton*
    codifica m in JSON e invialo al visore sulla porta 5005
    ▷ *un messaggio malformato viene segnalato e scartato,*
    ▷ *senza interrompere il ciclo*

Come mostra l'Algoritmo 4.3, lo script acquisisce sia la telemetria continua sia i riferimenti geometrici della traiettoria futura, scala le coordinate dividendo per mille e ricompone il JSON prima della trasmissione verso la porta 5005.

Per quanto riguarda il lato del visore, l'ascolto diretto del canale di rete all'interno del ciclo principale in Unity, cioè il Main Thread, potrebbe causare blocchi dell'esecuzione dovuti alle operazioni di I/O del socket. Anche piccoli fenomeni di oscillazioni temporali (jitter) nella trasmissione su canale wireless provocherebbero ritardi nel rendering del visore, innescando microscatti visivi e degradando la stabilità che viene percepita nella scena in Realtà Aumentata. Per mantenere l'indipendenza del ciclo grafico, attraverso un'architettura concorrente basata sul modello *Produttore-Consumatore*, la ricezione dei dati è stata separata. Questa struttura è stata implementata all'interno dello script `UDPReceiver.cs`. Quando viene inizializzata l'applicazione viene creato un thread asincrono dedicato (`receiveThread`), configurato per restare in ascolto continuo sulla porta 5005; nel momento in cui il thread acquisisce i pacchetti in arrivo, converte il flusso di byte nella stringa JSON corrispondente e la inserisce in una coda FIFO (`rawMessageQueue`). L'accesso a questa risorsa condivisa è racchiuso all'interno di una sezione critica protetta tramite costrutto `lock`, garantendo in questo modo la sincronizzazione tra il thread di rete e il thread di rendering ed evitando problemi di Interferenza concorrente, anche detta *race condition*.

Algoritmo 4.4 Ricezione su thread secondario, lato produttore.

Ingresso: socket in ascolto sulla porta 5005, con timeout di lettura
Uscita: messaggi JSON accodati in `rawMessageQueue`

```
1:  procedura ReceiveDataInBackground( )
2:      finché isRunning esegui
3:          attendi un datagramma con la chiamata bloccante Receive
4:          ▷ il thread resta fermo finché non arriva un pacchetto
5:          ▷ oppure finché non scade il timeout di lettura
6:          se è arrivato un datagramma allora
7:              s ← contenuto del datagramma, convertito in testo UTF-8
8:              ▷ sezione critica: il thread grafico non può accedere
9:              ▷ alla coda mentre il messaggio viene inserito
10:             acquisisci queueLock
11:                 accoda s in rawMessageQueue
12:             rilascia queueLock
13:         altrimenti se è scaduto il timeout di lettura allora
14:             ▷ eccezione SocketException
15:             torna all'inizio del ciclo e ricontrolla isRunning
16:         altrimenti se il socket è stato chiuso allora
17:             ▷ eccezione ObjectDisposedException
18:             esci dal ciclo: l'applicazione è in fase di chiusura
19:         altrimenti
20:             ignora l'errore e prosegui con il datagramma successivo
```

Come mostra l'Algoritmo 4.4, il controllo dell'esecuzione è affidato alla gestione mirata delle eccezioni generate dalla primitiva bloccante `Receive`, infatti l'acquisizione di `SocketException` consente di assorbire l'eventuale scadenza del timeout di lettura configurato sul socket, facendo rientrare l'esecuzione all'inizio del ciclo per rivalutare lo stato della variabile booleana `isRunning`; la gestione di `ObjectDisposedException`, invece, intercetta il rilascio del socket all'arresto dell'applicazione, forzando l'uscita immediata (`break`) e garantendo la chiusura controllata del thread secondario.

All'interno del metodo `Update`, richiamato a ogni fotogramma, l'applicazione prende ed esamina i messaggi accumulati nella coda. Se il sistema si limitasse a estrarre un solo pacchetto per frame, qualunque lieve asimmetria tra la frequenza di generazione del robot e il frame-rate del visore porterebbe ad un accumulo di dati arretrati, con un conseguente ritardo visivo crescente tra la posizione reale del manipolatore e la sfera virtuale.
Per evitare questa latenza accumulata, è stata adottata una strategia di svuo-

tamento completo della coda (`scartaTelemetriaArretrata`). A ogni ciclo, l'intero contenuto di `rawMessageQueue` viene trasferito in una lista locale all'interno di una singola operazione protetta. Durante l'elaborazione del blocco, il sistema applica un trattamento diverso in base alla natura del messaggio: per i pacchetti di telemetria, contenenti lo stato istantaneo del robot, ogni campione che precede l'ultimo ricevuto è privo di utilità e viene scartato, aggiornando la variabile di stato unicamente con il dato più recente; per i messaggi di previsione della traiettoria, al contrario, essendo essi eventi discreti dipendenti dalla pianificazione del moto, nessun pacchetto viene eliminato e i dati vengono trasferiti in sequenza nella coda dedicata `trajectoryQueue`.

Algoritmo 4.5 Svuotamento della coda, lato consumatore.

Ingresso: coda `rawMessageQueue` riempita dal thread di ricezione
Uscita: posa e forze aggiornate; previsioni accodate in `trajectoryQueue`

procedura ProcessNetworkMessages( )
    ▷ *eseguita dal thread grafico a ogni fotogramma, all'interno di* `Update`
    acquisisci `queueLock`
        $B \leftarrow$ tutti i messaggi presenti in `rawMessageQueue`
        svuota `rawMessageQueue`
    rilascia `queueLock`
    ▷ *il lucchetto copre solo il trasferimento: la decodifica,*
    ▷ *più lenta, avviene fuori dalla sezione critica*
    se B è vuoto **allora**
        termina
    $u \leftarrow$ nessuna telemetria
    per ogni messaggio $b \in B$, in ordine di arrivo **esegui**
        $m \leftarrow$ decodifica JSON di b tramite `JsonUtility`
        se m è un messaggio di telemetria **allora**
            $u \leftarrow m$
            ▷ *sovrascrive la precedente: conta solo la più recente*
        altrimenti se m è un messaggio di previsione **allora**
            accoda m in `trajectoryQueue`
            ▷ *le previsioni sono eventi discreti: nessuna viene scartata*
    se è stata ricevuta almeno una telemetria u **allora**
        posizione corrente $\leftarrow$ posizione di u, convertita tramite **S**
        forza corrente $\leftarrow$ forza di u
        aggiorna la sfera olografica e l'interfaccia

La deserializzazione dei dati JSON ricevuti viene gestita tramite la classe di sistema `JsonUtility`. Una volta individuato l'ultimo messaggio valido, come

mostrato nell'Algoritmo 4.5, la posizione del punto operativo viene convertita nel sistema di riferimento utilizzato da Unity attraverso il metodo `ToVector3()` della struttura `JsonPoint`. In questo passaggio viene applicata la trasformazione tra i sistemi di coordinate descritta nel paragrafo precedente.
Parallelamente, le componenti cartesiane della forza (F_x, F_y, F_z), espresse in newton e ricavate dalle misure dei sensori di coppia presenti sui giunti del manipolatore, vengono memorizzate nella variabile `currentRobotForce`. Le coordinate della posizione vengono utilizzate per determinare la posizione dell'ologramma rispetto all'oggetto `RobotRoot`, mentre i valori delle forze vengono visualizzati nei relativi campi numerici dell'interfaccia utente.

4.6 Rendering del Digital Twin, Tracciamento della Flangia e Traiettorie Predittive

Gli elementi principali che compongono il Digital Twin nell'ambiente immersivo sono due: la rappresentazione in tempo reale della flangia del robot e la proiezione anticipata delle traiettorie programmate. Questi due elementi si trovano entrambi, nell'Hierarchy di Unity, al di sotto di `RobotRoot`, in questo modo viene garantito che ogni aggiornamento di posizione avvenga nello spazio locale rispetto allo zero macchina calibrato.
Per quanto riguarda la visualizzazione della flangia del robot, essa è assegnata a un ologramma sferico, il `RobotMarker`. Per evitare di creare ambiguità visiva all'interno della scena aumentata, è necessario distinguere il `RobotMarker` dall'indicatore a colorazione variabile utilizzato nella fase di calibrazione: quest'ultimo è un oggetto solido alla base del robot che varia il suo colore (rosso, giallo, verde) per segnalare lo stato del campionamento ottico, mentre, il marcatore della flangia del robot mantiene una visualizzazione grafica costante ed è assegnato unicamente al riscontro geometrico del moto.
Il marcatore, sul piano cinematico, ha il compito di tracciare la posizione della **flangia Meccanica** del robot e non il *Tool Center Point*. A livello software, all'interno del controllore, l'applicazione Sunrise estrae le coordinate mediante il metodo `robot.getFlange()`. Non disponendo di una calibrazione metrologica specifica per l'utensile utilizzato, si è scelto di fare riferimento alla flangia, dato che la sua cinematica è definita in maniera specifica dal modello geometrico fornito dal costruttore.
L'aggiornamento della posizione avviene all'interno del metodo `Update` con il ritmo del ciclo di rendering grafico del visore. Per ogni fotogramma, l'applicazione utilizza l'ultimo campione valido reso disponibile dal flusso di rete

a 50 Hz, assegnandolo alla proprietà `localPosition` del marcatore. Questa assegnazione locale garantisce che la sfera olografica segua con continuità lo spostamento fisico della flangia visibile attraverso il passthrough, mantenendo in questo modo la coerenza geometrica a meno della latenza di trasmissione e campionamento.

L'applicazione, oltre a gestire l'inseguimento da parte della sfera olografica della flangia del robot, parallelamente visualizza l'intento cinematico del robot estraendo i messaggi di previsione presenti nella coda `trajectoryQueue`. La rappresentazione del percorso è implementata tramite il componente `LineRenderer` di Unity, configurato per operare nello spazio locale del manipolatore con `useWorldSpace = false`.
Il sistema gestisce due tipologie di moto, cioè un moto lineare, dove il segmento unisce direttamente il punto di partenza e il punto di arrivo, e un moto circolare, dove la geometria dell'arco tridimensionale viene ricostruita tramite tre coordinate nello spazio, cioè partenza, punto ausiliario intermedio e arrivo. All'interno di un moto circolare, tre punti non allineati individuano una circonferenza nello spazio tridimensionale, ma su di essa esistono due percorsi possibili per collegare il punto iniziale e quello finale, cioè l'arco minore e l'arco maggiore. Per questo motivo, il punto ausiliario assume un ruolo fondamentale nella scelta del percorso, infatti, esso indica quale porzione di cerchio debba essere descritta e in quale verso, replicando in maniera fedele il comportamento dell'istruzione di moto circolare del controllore KUKA.

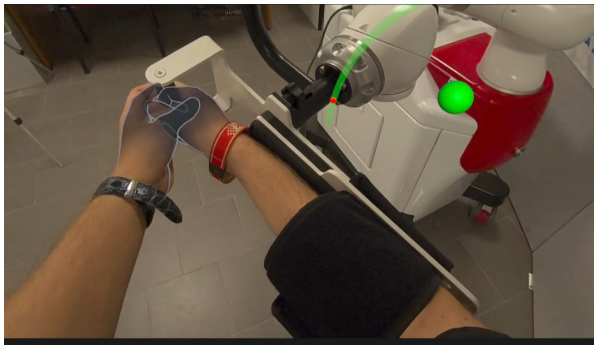


Figura 4.4: Visualizzazione del Digital Twin e della Traiettoria Predittiva

Algoritmo 4.6 Ricostruzione dell'arco circolare a partire da tre punti.

Ingresso: punto di partenza $\mathbf{p}_1$, punto ausiliario $\mathbf{p}_2$, punto di arrivo $\mathbf{p}_3$
Uscita: sequenza di punti dell'arco da assegnare al `LineRenderer`

funzione DrawThreePointArc($\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$)
  $\mathbf{v}_1 \leftarrow \mathbf{p}_2 - \mathbf{p}_1$, $\mathbf{v}_2 \leftarrow \mathbf{p}_3 - \mathbf{p}_1$
  $\mathbf{n} \leftarrow \mathbf{v}_1 \times \mathbf{v}_2$
  ▷ *vettore normale al piano che contiene i tre punti*
  se $\|\mathbf{n}\|$ è trascurabile rispetto a $\|\mathbf{v}_1\|\,\|\mathbf{v}_2\|$ **allora**
    ▷ *punti allineati: nessuna circonferenza, si disegna la spezzata*
    restituisci $(\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3)$
  ▷ *centro della circonferenza passante per i tre punti e suo raggio*
  $\mathbf{c} \leftarrow \mathbf{p}_1 + \dfrac{(\mathbf{n} \times \mathbf{v}_1)\,\|\mathbf{v}_2\|^2 + (\mathbf{v}_2 \times \mathbf{n})\,\|\mathbf{v}_1\|^2}{2\,\|\mathbf{n}\|^2}$
  $r \leftarrow \|\mathbf{p}_1 - \mathbf{c}\|$
  ▷ *base ortonormale nel piano dell'arco, con* $\mathbf{u}$ *rivolto verso la partenza*
  $\mathbf{u} \leftarrow (\mathbf{p}_1 - \mathbf{c})/r$, $\mathbf{w} \leftarrow (\mathbf{n}/\|\mathbf{n}\|) \times \mathbf{u}$
  $\alpha_2 \leftarrow$ AngoloNelPiano($\mathbf{p}_2$), $\alpha_3 \leftarrow$ AngoloNelPiano($\mathbf{p}_3$)
  ▷ *se l'arrivo precede l'ausiliario, l'arco corretto è il complementare,*
  ▷ *percorso nel verso opposto: l'ampiezza diventa negativa*
  se $\alpha_3 < \alpha_2$ **allora**
    $\Delta \leftarrow \alpha_3 - 2\pi$
  altrimenti
    $\Delta \leftarrow \alpha_3$
  ▷ *discretizzazione: un segmento ogni 0,035 rad circa, da 8 a 120*
  $k \leftarrow \min\big(\max(\lceil |\Delta|/0{,}035 \rceil,\ 8),\ 120\big)$
  per $i = 0, \ldots, k$ **esegui**
    $\varphi \leftarrow \Delta \cdot i/k$
    $\mathbf{x}_i \leftarrow \mathbf{c} + r\,(\cos\varphi\,\mathbf{u} + \sin\varphi\,\mathbf{w})$
  restituisci $(\mathbf{x}_0, \ldots, \mathbf{x}_k)$

funzione AngoloNelPiano($\mathbf{x}$)
  ▷ *coordinata angolare di* $\mathbf{x}$ *attorno al centro, misurata a partire da* $\mathbf{u}$
  $\mathbf{d} \leftarrow \mathbf{x} - \mathbf{c}$
  $\alpha \leftarrow \operatorname{atan2}(\mathbf{d} \cdot \mathbf{w},\ \mathbf{d} \cdot \mathbf{u})$
  restituisci α riportato nell'intervallo $[0, 2\pi)$

Come mostrato nell'Algoritmo 4.6, l'algoritmo calcola il vettore normale al piano dell'arco tramite il prodotto vettoriale $\mathbf{n} = \mathbf{v}_1 \times \mathbf{v}_2$. Quando viene verificato che i punti sono non collineari, vengono determinati centro e raggio della circonferenza, per poi costruire una base ortonormale $\{\mathbf{u}, \mathbf{w}\}$ che giace

sul piano del moto. La funzione `AngoloNelPiano` proietta ogni punto dell'arco e ne restituisce la coordinata angolare, cioè l'angolo misurato attorno al centro a partire dalla direzione del punto iniziale, assunta come riferimento. Il calcolo viene effettuato dall'arcotangente a due argomenti, `Mathf.Atan2`, che a differenza dell'arcotangente ordinaria riceve le due componenti della proiezione in maniera separata e ne considera i segni, restituendo così un valore univoco sull'intero giro anziché solo sull'intervallo $(-\pi/2, \pi/2)$. Questo risultato viene normalizzato nell'intervallo $[0, 2\pi)$. Per costruzione, il punto di partenza ha coordinata angolare nulla, mentre il punto ausiliario e quello di arrivo assumono valori crescenti nel verso determinato dalla normale al piano. Facendo il confronto fra queste due coordinate viene stabilito quale dei due archi deve essere percorso: se il punto di arrivo precede angolarmente quello ausiliario, l'arco corretto non è quello che si incontra procedendo in senso crescente, ma il suo complementare, e sottraendo 2π all'ampiezza si ottiene un valore negativo che corrisponde a percorrere la circonferenza nel verso opposto. È questa condizione a distinguere l'arco corretto dal suo complementare [22].

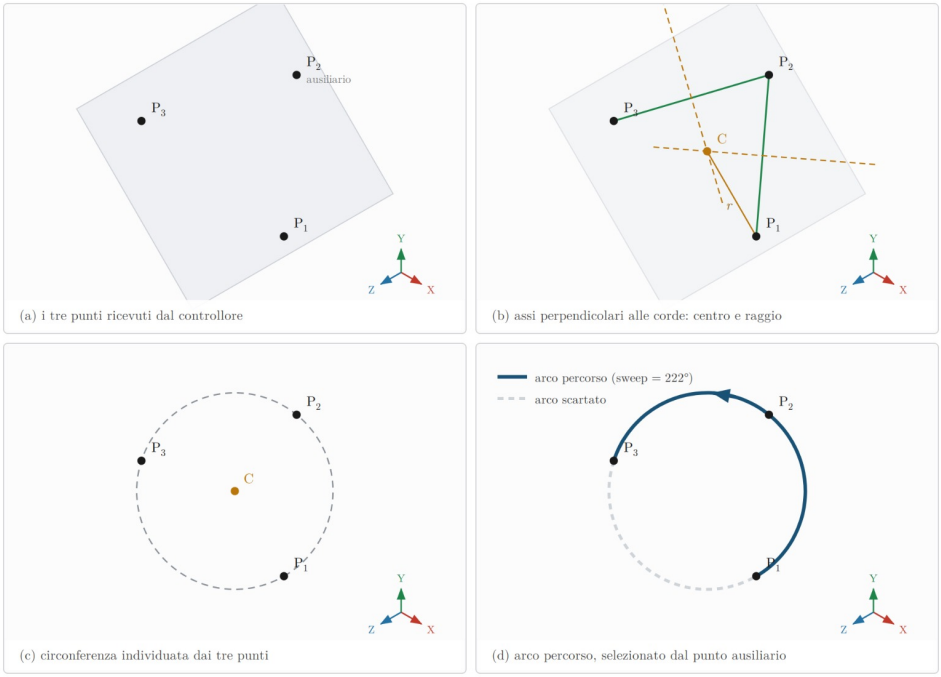


Figura 4.5: Schematizzazione della Costruzione dell'Arco di Circonferenza

Quando la traiettoria rappresentata viene percorsa, per non occupare il campo visivo dell'operatore, essa viene rimossa dall'ambiente virtuale. Il sistema misura quindi a ogni fotogramma la distanza fra la posizione della flangia e il punto di arrivo previsto: quando questa scende sotto i 5 cm il moto si considera concluso e la linea comincia a svanire, perdendo opacità in modo

graduale nell'arco di 0,8 s prima di essere rimossa. La dissolvenza, oltre a evitare la comparsa e la scomparsa improvvisa delle geometrie, lascia il tempo di cogliere il passaggio da una traiettoria alla successiva.

4.7 Interfaccia Utente (HMI) e Tracciamento delle Mani

In questo paragrafo vengono descritte le motivazioni e le modalità di sviluppo relative alla scelta di utilizzare l'*Hand Tracking* del visore Meta Quest 3 e l'interfaccia utente visualizzata nell'applicazione.
Durante le operazioni che vengono effettuate con l'applicazione sviluppata, l'impiego dei controller fisici tradizionali costringerebbe l'operatore a impegnare le mani, limitandone la capacità di intervenire sul manipolatore robotico, maneggiare componenti di lavoro o agire sui vari controller e dispositivi di controllo del sistema. Per questo motivo, l'interfaccia uomo-macchina (HMI) è stata realizzata sfruttando il tracciamento ottico delle mani (*Hand Tracking*) integrato nel Meta Quest 3, consentendo, in questo modo, un controllo privo di dispositivi esterni.
L'interazione delle mani dell'operatore con l'HMI è basata sul gesto del *pinch*, cioè il movimento che le dita compiono accostando la punta dell'indice e quella del pollice. Questo gesto funge da interruttore per mostrare o nascondere il pannello informativo, riducendo l'ingombro visivo quando l'operatore necessita della piena visibilità sull'area di lavoro. Per prevenire delle attivazioni non volute dovute al tremolio naturale della mano o a micro oscillazioni nel riconoscimento ottico, l'acquisizione del gesto comprende un filtro temporale di riconoscimento (`gesturecooldown`), che impone un'attesa minima di 0.5 s tra commutazioni consecutive.

Algoritmo 4.7 Rilevamento del pinch e commutazione dell'interfaccia.

Ingresso: stato della mano destra, fornito dall'hand tracking
Uscita: pannello HMI mostrato o nascosto, una sola volta per ogni pinch

```
1:  procedura HandleInputAndPinch( )
2:      ▷ il tempo di attesa residuo diminuisce a ogni fotogramma
3:      se cooldownTimer > 0 allora
4:          cooldownTimer ← cooldownTimer − Δt
5:      se la mano destra è tracciata in modo affidabile allora
6:          pinchAttuale ← indice e pollice a contatto
7:          ▷ le dita si toccano ora ma non si toccavano al fotogramma
8:          ▷ precedente, e l'attesa minima fra due comandi è trascorsa
9:          se pinchAttuale e non isPinching e cooldownTimer ≤ 0 allora
10:             mostra o nascondi il pannello HMI
11:             cooldownTimer ← gesturecooldown
12:             ▷ nuova attesa di 0,5 s prima del comando successivo
13:         ▷ stato memorizzato per il confronto al fotogramma successivo
14:         isPinching ← pinchAttuale
```

Come descritto nell'Algoritmo 4.7, la condizione `pinchAttuale && !isPinching` fa in modo che il comando venga eseguito solo nell'istante in cui le dita si toccano, e non per tutto il tempo in cui le dita restano unite. Senza questo controllo l'HMI comparirebbe e scomparirebbe decine di volte al secondo finché il gesto resta attivo. A ogni pinch corrisponde esattamente una commutazione, qualunque sia la sua durata.
Per quanto riguarda la visualizzazione dei parametri, essa è affidata all'HMI (`hmiCanvas`), implementata tramite un componente *Canvas* in modalità *World Space*. Questo pannello risulta essere vincolato fisicamente al lato del dispositivo robotico, facendo riferimento all'origine degli assi del sistema.
All'interno dell'Hierarchy di Unity, il canvas è configurato come figlio diretto di `RobotRoot`, questo perché il pannello non deve sovrapporsi ad alcun elemento fisico del manipolatore e non necessita quindi della correzione di offset descritta nel paragrafo 4.4. Tramite questa impostazione viene garantito che la posizione nello spazio rimanga rigidamente ancorata alla base del manipolatore, come detto precedentemente, preservando l'acquisizione metrica ottenuta tramite il marker ottico. Allo stesso tempo, per assicurare una lettura dell'HMI ottimale da parte dell'operatore da qualsiasi posizione nello spazio esso si trovi, l'orientamento del pannello visivo viene svincolato da quello della base tramite l'ausilio dello script `BillboardUI.cs`.

Algoritmo 4.8 Orientamento del pannello HMI verso l'operatore.

Ingresso: posizione del pannello $\mathbf{p}_H$ e della testa dell'operatore $\mathbf{p}_C$
Uscita: pannello rivolto all'operatore e sempre verticale

```
1: procedura LateUpdate( )
2:     ▷ eseguita a ogni fotogramma, dopo che la posa della testa è stata
3:     ▷ aggiornata e prima che la scena venga disegnata
4:     d ← p_C − p_H
5:     ▷ direzione dal pannello verso l'operatore
6:     d_y ← 0
7:     ▷ proiezione sul piano orizzontale: il pannello ruota soltanto
8:     ▷ attorno all'asse verticale e non si inclina mai
9:     se ‖d‖ è prossima a zero allora
10:        ▷ operatore sulla verticale del pannello: direzione indefinita
11:        termina senza modificare l'orientamento
12:    orienta l'asse z locale del pannello lungo −d
13:    ▷ segno invertito: il lato leggibile del pannello è opposto al suo asse z
```

Come mostrato nell'Algoritmo 4.8, il calcolo viene eseguito in `LateUpdate`, cioè al termine della fase di aggiornamento e prima che la scena venga disegnata, infatti, in questo momento la posizione della testa è già stata risolta per il fotogramma corrente, e il pannello si orienta senza il ritardo che si otterrebbe anticipando l'operazione. L'annullamento della componente verticale della direzione limita la rotazione al solo asse verticale, dando luogo a un billboard di tipo cilindrico: il cruscotto non resta perpendicolare alla linea di vista quando l'operatore osserva dall'alto o dal basso, ma in cambio non si inclina mai, conservando la verticalità come riferimento stabile. L'inversione di segno del vettore, infine, è necessaria perché in Unity il lato leggibile di un pannello d'interfaccia è rivolto in verso opposto al suo asse z locale: orientando quest'ultimo direttamente verso l'osservatore, il pannello gli mostrerebbe il retro e il testo apparirebbe speculare. Invertire il vettore equivale a ruotare il pannello di 180° attorno alla verticale, cioè a una rotazione propria, che non va confusa con il cambio di chiralità discusso nel Paragrafo 4.2. Il risultato complessivo è un pannello che eredita dal RobotRoot la sola posizione, restando ancorato alla macchina mentre ruota per presentarsi all'operatore.
Per quanto riguarda il contenuto della tabella, essa organizza le informazioni in due sezioni distinte, cioè: la prima, dove vengono riportate le coordinate cartesiane istantanee della flangia (X, Y, Z) espresse in metri, la seconda, dove vengono trascritte le componenti scalari della forza esterna (F_x, F_y, F_z) espresse in newton. I valori mostrati in tabella sono quelli già convertiti secondo la matrice descritta nel Paragrafo 4.4, e sono quindi espressi nel sistema di

riferimento del RobotRoot, con l'asse Y verticale, quindi non coincidono con i numeri leggibili sullo smartPAD del controllore, che adotta la terna nativa del robot. L'aggiornamento dei campi di testo avviene soltanto quando il pannello è visibile, così da non eseguire formattazioni inutili mentre l'interfaccia è nascosta.

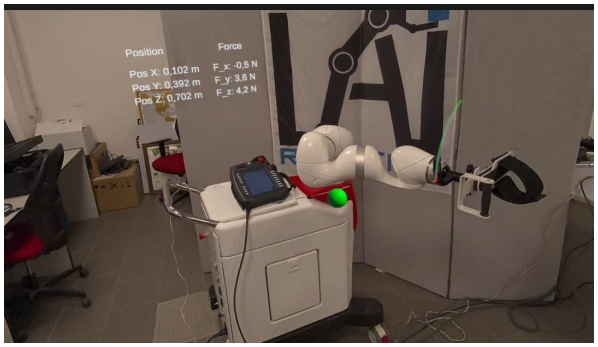


Figura 4.6: Visualizzazione dello Human-Machine Interface all'Interno dell'Applicazione in Realtà Aumentata

Capitolo 5

Validazione e Criticità Riscontrate

In questo capitolo verranno analizzati i test effettuati riguardo i diversi aspetti relativi al progetto durante le attività di verifica, e verranno descritte alcune problematiche principali incontrate durante questa fase. In particolare sono stati verificati: l'integrità della trasmissione di rete, la stabilità dell'ancoraggio spaziale e la coerenza della sovrapposizione olografica. L'attività di verifica si compone di due fasi, cioè una prima fase di collaudo condotta interamente via software, con lo scopo di gestire la logica di elaborazione in modo indipendente e prevenire possibili rischi per l'hardware, e una fase di validazione sperimentale sul manipolatore KUKA LBR med 14 R820.

5.1 Ambiente di Simulazione e Collaudo della Catena Dati

Lavorare con sistemi a Realtà Aumentata insieme a dispositivi robotici come il KUKA, impone all'operatore diversi vincoli di lavoro. Per questo motivo, per evitare problematiche di sicurezza, e soprattutto per non dipendere dalla presenza del dispositivo KUKA nelle prime fasi di sviluppo, il flusso di dati è stato testato disaccoppiando l'applicazione dal manipolatore.
Per quanto riguarda l'ambiente di sviluppo in Unity, la verifica dell'interfaccia utente e del tracciamento delle mani è stata condotta alternando sessioni di esecuzione tramite *Meta Quest Link*, in questo caso sfruttando lo streaming cablato per velocizzare il debug, con l'installazione di **Standalone Build** dell'applicazione direttamente sul visore in formato `.apk`. In questo modo è stato possibile testare le interazioni con i gesti basate sul componente `OVRHand`, verificare il filtro di acquisizione del gesto di *pinch* e verificare il corretto orientamento dell'HMI a un grado di libertà implementato nello script `BillboardUI.cs`.
Per la verifica del ricevitore di rete e la ricostruzione grafica delle traiettorie,

è stato utilizzato lo script di emulazione `terzo_simulatore_kuka_forze.py`. Questo script non emula solo il controllore Sunrise, ma sostituisce l'intera catena formata da robot e calcolatore intermedio: esso genera pacchetti JSON con coordinate cartesiane già espresse in metri e li trasmette direttamente verso la porta 5005 del visore alla frequenza di 50 Hz. Il simulatore alterna in modo ciclico l'invio di un pacchetto di previsione relativo al moto circolare e uno relativo alla previsione di un moto lineare.

5.2 Verifica dell'Ancoraggio Spaziale e Calibrazione dell'Offset

L'utilizzo dell'unione del manipolatore robotico KUKA e il mondo virtuale del visore Meta Quest 3 ha richiesto la convalida dell'ancoraggio spaziale basato sul QR CODE come descritto nel paragrafo 4.3. Il sistema di riconoscimento è affidato alle API native del *Mixed Reality Utility Kit* (MRUK). Tramite questa soluzione è possibile affidare l'estrazione delle caratteristiche al sistema operativo del visore. Sarebbe possibile accedere direttamente ai flussi video delle telecamere tramite le *Passthrough* Camera API, basate sulle interfacce `Camera2` di Android, previa abilitazione della modalità sviluppatore sul Meta Quest 3; l'utilizzo di MRUK, tuttavia, evita di introdurre una libreria di visione artificiale aggiuntiva, riducendo il costo computazionale complessivo.
L'allineamento del sistema di coordinate del visore e quello del sistema robotico dipende in maniera stretta dalla stima del vettore di offset dovuto alla distanza del baricentro del marker e l'origine degli assi di riferimento del robot. Nell'applicazione realizzata, questo valore è stato inizialmente definito tramite una misura metrica manuale effettuata direttamente sulla base del manipolatore dove veniva posizionato il marker; successivamente è stato affinato in diverse iterazioni mediante un riscontro visivo in modalità passthrough, confrontando la coincidenza spaziale tra il marcatore olografico e i riferimenti fisici della flangia.
La procedura di calibrazione ha confermato l'efficacia dell'uso del filtro di acquisizione visivo a media temporale su trenta campioni consecutivi. L'intero processo di convergenza richiede tra gli 1.5 e 2 secondi, infatti, questo intervallo di tempo comprende la stabilizzazione del campo visivo da parte dell'operatore e l'eventuale azzeramento del buffer di campionamento nel caso in cui vengano superate le soglie di tolleranza impostate sulla dispersione geometrica.

5.3 Analisi della Sincronizzazione Geometrica e Problematiche Riscontrate

La fase di collaudo finale del progetto ha riguardato la corrispondenza geometrica tra il movimento reale del manipolatore e quello del Digital Twin, questo, comandando al KUKA traiettorie tramite lo smartPAD e osservando la sovrapposizione ottica in passthrough tra la flangia meccanica e la sfera olografica. In particolare in questa fase sono emerse diverse criticità principali, tra cui il problema dell'**Inversione di Chiralità**, infatti nelle prime versioni del convertitore geometrico, un'errata combinazione tra scambio degli assi cartesiane inversione di segno produceva una matrice con determinante positivo. Essendo una rotazione propria nello spazio, questa matrice non era in grado di mappare la terna destrorsa del controllore nella terna sinistrorsa di Unity, causando uno spostamento specchiato della sfera olografica lungo gli assi. L'introduzione della matrice di riflessione con $det = -1$ ha risolto il problema, dando coerenza di moto lungo le tre direzioni.
Un'altra problematica incontrata riguarda la **Gestione del Jitter di Rete su Canale Wireless**, infatti nelle prove di trasmissione via Wi-Fi sono state registrate delle fluttuazioni fisiologiche nel tempo di passaggio dei pacchetti. Grazie all'architettura software a due livelli, è stato evitato qualsiasi accumulo di ritardo nel buffer data la logica di scarto della telemetria arretrata, in questo modo, il motore grafico è riuscito a mantenere il target di rendering senza congelamenti o disallineamenti temporali percepibili.
La figura 5.1 mostra l'effetto di mappatura sulla sovrapposizione.

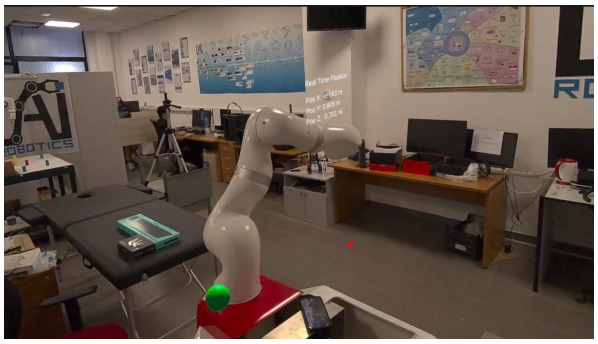
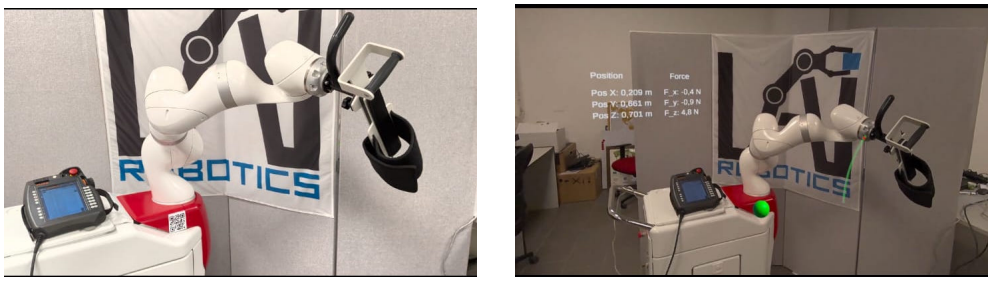


Figura 5.1: Visualizzazione dell'Errore di Sincronizzazione dei Sistemi di Riferimento dei Due Domini

Capitolo 6

Conclusioni e Sviluppi Futuri

In questo capitolo finale Il lavoro di tesi svolto ha portato alla realizzazione di un sistema in Realtà Aumentata in grado di sovrapporre, all'immagine reale di un manipolatore collaborativo osservata in passthrough, la posizione istantanea della sua flangia e la traiettoria che essa sta per percorrere. Per fare in modo che la sovrapposizione risultasse significativa sono stati messi in comunicazione due mondi diversi fra loro che descrivono lo spazio circostante in maniera completamente differente; gran parte dell'attività di sviluppo si è concentrata su questo aspetto.
La struttura del progetto collega l'applicazione Java in esecuzione sul controllore Sunrise a un calcolatore che funge da ponte, e da questo, all'applicazione sviluppata in Unity installata sul visore, mediante pacchetti JSON trasmessi su socket UDP. Lungo questa struttura si è necessitato distinguere due trasformazioni matematiche distinte, cioè una riflessione fissa, a determinante negativo, con il compito di convertire la terna destrorsa del robot nella terna sinistrorsa di Unity, e una rototraslazione propria, stimata a ogni sessione tramite il riconoscimento di un QR CODE fisico posto alla base del manipolatore, che posiziona l'origine del sistema di riferimento cartesiano del robot all'interno dello spazio tracciato dal visore. La separazione di queste due trasformazioni, concentrata nel nodo *RobotRoot*, ha reso la struttura indipendente dalla posizione fisica del robot nell'ambiente, caratteristica necessaria essendo il manipolatore posizionato su un carrello mobile. A livello di visualizzazione sono state implementate la rappresentazione continua della posizione della flangia in tempo reale, la ricostruzione geometrica delle traiettorie predittive, sia lineari che circolari, e un'interfaccia di visualizzazione dati ancorata alla macchina che riporta posizione e forze stimate.
La figura 6.1 mostra il sistema in funzione sul manipolatore reale, osservato nel medesimo istante dall'esterno e dall'interno del visore.



(a) Ripresa esterna. (b) Vista dal visore.

Figura 6.1: Il sistema in funzione sul manipolatore KUKA LBR Med: a sinistra la scena osservata dall'esterno, a destra la stessa scena vista attraverso il Meta Quest 3, con la sfera olografica sovrapposta alla flangia e la traiettoria prevista disegnata nello spazio.

6.1 Sviluppi Futuri

Tra i limiti più evidenti del progetto c'è la procedura di registrazione spaziale, infatti, la distanza tra il marker fisico e l'origine effettiva del dispositivo robotico è stata determinatoria per via manuale e affinata visivamente. Questa procedura risulta essere sufficiente per effettuare la sincronizzazione dei due elementi, ma risulta essere priva di una caratterizzazione metrologica. Un miglioramento, riguardo questo aspetto, sarebbe quello di mandare il manipolatore robotico su un insieme di punti noti e risolvere il problema di allineamento fra le coincidenze, ottenendo anche un valore che permette di valutare quanto siano accurate le stime effettuate, misurando la differenza tra i valori stimati e quelli reali. Questa stessa procedura consentirebbe di stimare la posizione del marker rispetto alla base in modo automatico, eliminando così la necessità di intervenire manualmente a ogni riposizionamento del carrello.
Per quanto riguarda la rappresentazione che viene mostrata nell'applicazione, essa si limita alla sola posizione cartesiana. L'aggiunta dell'orientamento, messa a disposizione dal controllore tramite gli angoli di Eulero, permetterebbe di visualizzare l'assetto dell'utensile montato sulla flangia, fatto che permetterebbe il passaggio dal tracciamento della flangia a quello del Tool Center Point, avendo una descrizione dell'utensile nel programma del manipolatore.
Tra le varie migliorie che possono essere effettuate, la più importante è sicuramente rappresentata dalla possibilità di poter interagire e controllare il sistema robotico all'interno dell'ambiente virtuale. Nella configurazione attuale il flusso informatico è unidirezionale, infatti il contenuto virtuale descrive lo stato della macchina senza influenzarlo. Consentire all'operatore di intervenire dal visore comporterebbe il passaggio ad un flusso di dati bidirezionale e quindi a un sistema di comando robotico in Realtà Aumentata.
Un'ultima evoluzione possibile riguarda la piattaforma Hardware. L'ergonomia del sistema rappresenta uno degli aspetti più importanti per un'applicazione

destinata a sessioni di lavoro prolungate, soprattutto in ambito biomedico. Il peso del dispositivo rappresenta sotto questo aspetto il principale fattore limitante. Risulta quindi significativa l'evoluzione del modello del visore annunciata da Meta con i nuovi **Meta VR Glasses**, presentati nel settembre 2026 e attesi sul mercato nella primavera 2027. Il dispositivo avrà una massa di circa 100 g, pari a circa un quinto del peso del Meta Quest 3, infatti, questo è stato reso possibile trasferendo le unità di calcolo e alimentazione in modulo esterno collegato via USB-C [26]. Una riduzione di peso di tale portata renderebbe sostenibile l'impiego continuativo del sistema per un tempo prolungato con maggiore facilità. L'effettiva trasferibilità dell'applicazione su tale piattaforma resta però da verificare, infatti, al momento non risultano dichiarate le caratteristiche del passthrough a colori, né se saranno disponibili interfacce di comprensione della scena e di riconoscimento di eventuali marker.



Figura 6.2: Visualizzazione del nuovo Meta VR Glasses [26]

Bibliografia

[1] P. Milgram and F. Kishino, "A taxonomy of mixed reality visual displays," *IEICE TRANSACTIONS on Information and Systems*, vol. 77, no. 12, pp. 1321–1329, 1994.

[2] R. T. Azuma, "A survey of augmented reality," *Presence: teleoperators & virtual environments*, vol. 6, no. 4, pp. 355–385, 1997.

[3] C. N. Nwokedi, O. S. Soyege, O. D. Balogu, A. Y. Mustapha, B. O. Tomoh, A. O. Mbata, and D. R. Iguma, "Virtual reality (vr) and augmented reality (ar) in medicine: A review of clinical applications," *International Journal of Scientific Research in Science, Engineering and Technology (IJSRSET)*, vol. 11, pp. 438–449, Nov-Dec 2024.

[4] P. Milgram, H. Colquhoun, *et al.*, "A taxonomy of real and virtual world display integration," *Mixed reality: Merging real and virtual worlds*, vol. 1, no. 1999, pp. 1–26, 1999.

[5] KUKA Deutschland GmbH, *System Software KUKA Sunrise.OS Med 2.7, KUKA Sunrise.Workbench Med 2.7 Operating and Programming Instructions for System Integrators Instructions for Use.* Augsburg, Germany, 2025. Version: BA KUKA Sunrise.OS Med 2.7 V3, Issued: 19.12.2025.

[6] KUKA AG, "Robot medicale lbr med," 2026. Pagina presentazione Kuka LBR med.

[7] Unity Technologies, "Motion-to-photon latency," 2026.

[8] Meta Platforms Inc., "Developer analytics: An introduction - performance metrics." `https://developers.meta.com/horizon/blog/developer-analytics-an-introduction/`, 2021. Documentazione ufficiale Meta sulle metriche di performance, frame rate e prevenzione di stuttering e judder in ambiente visivo.

[9] Meta Platforms Inc., *Meta Quest 3 Manual*, 2023. Manuale d'uso e configurazione hardware.

[10] J. Pirker, E. Loria, S. Safikhani, A. Künz, and S. Rosmann, "Immersive virtual reality for virtual and digital twins: A literature review to identify state of the art and perspectives," in *2022 IEEE conference on virtual reality and 3D user interfaces abstracts and workshops (VRW)*, pp. 114–115, IEEE, 2022.

[11] A. Künz, S. Rosmann, E. Loria, and J. Pirker, "The potential of augmented reality for digital twins: A literature review," in *2022 IEEE conference on virtual reality and 3D user interfaces (VR)*, pp. 389–398, IEEE, 2022.

[12] K. D. GmbH, *Robot Option Media Flange For Product Family LBR iiwa Assembly and Operating Instructions*, 2021. ssued: 20.10.2021 Option Media Flange V10 KUKA Roboter GmbH.

[13] Meta Platforms Inc., "Meta quest 3 - specifiche tecniche." `https://www.meta.com/it/quest/quest-3/`, 2026. [Online].

[14] Meta Platforms Inc., "Come usare il pass-through su meta quest." `https://www.meta.com/it-it/help/quest/617966963105359/`, 2026. [Online; consultato il 9 agosto 2026].

[15] U. technologies, "Unity overview," 2015. Consultato il 04/09/2026.

[16] Meta, "Meta xr sdks for unity," 2025. Consultato il 05/09/2026.

[17] J. Postel, "User datagram protocol," tech. rep., RFC Editor, 1980.

[18] J. Postel, "Rfc 793: Transmission control protocol," 1981.

[19] M. Völter, M. Kircher, U. Zdun, and M. Englbrecht, "Patterns for asynchronous invocations in distributed object frameworks," in *Proceedings of the 8th European Conference on Pattern Languages of Programs (EuroPLoP '03)*, pp. 269–284, Hillside Group, 2003.

[20] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*. Pearson, 8th ed., 2021.

[21] L. Sciavicco and B. Siciliano, *Modelling and control of robot manipulators*. Springer Science & Business Media, 2012.

[22] E. Lengyel, *Mathematics for 3D game programming and computer graphics*. Course Technology Press, 2011.

[23] Meta, "Mixed reality utility kit - overview," 2026.

[24] F. L. Markley, Y. Cheng, J. L. Crassidis, and Y. Oshman, "Averaging quaternions," *Journal of Guidance, Control, and Dynamics*, vol. 30, no. 4, pp. 1193–1197, 2007.

[25] Unity Technologies, “Quaternion and euler rotations in unity,” 2025.

[26] Meta, “Presentazione dei meta vr glasses.”

Un sincero ringraziamento al Prof. Filippo Arrichiello
e al Prof. Daniele Di Vito,
per la costante disponibilità, i preziosi consigli
e la guida attenta offerta durante
lo sviluppo di questo lavoro di tesi.